

Chulalongkorn University

Submit and Pray

Teetat Thamronglak
Thawin Tengamnuay
Nantapong Ruangsuksriwong

2026-10-26

1 Contest 1
2 Mathematics 1
3 Data structures 3
4 Number Theory 4
5 Combinatorial 6
6 Polynomials 7
7 Convolutions 10
8 Graph 10
9 Tree 11
10 Flows 12
11 String 16
12 Linear Algebra 17
13 Linear Programming 18
14 Geometry 19
15 Miscellaneous 23

Contest (1)

template.hpp 52 lines

```
#pragma once
#include<bits/stdc++.h>
#include<ext/pb_ds/assoc_container.hpp>
#include<ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

#define pb push_back
#define eb emplace_back

#define ALL(a) a.begin(),a.end()
#define RALL(a) a.rbegin(),a.rend()
#define SORT(a) sort(ALL(a))
#define RSORT(a) sort(RALL(a))
#define REV(a) reverse(ALL(a))
#define UNI(a) a.erase(unique(ALL(a)),a.end())
#define SZ(a) (int)(a.size())
#define LB(a,x) (int)(lower_bound(ALL(a),x)-a.begin())
#define UB(a,x) (int)(upper_bound(ALL(a),x)-a.begin())
#define MIN(a) *min_element(ALL(a))
#define MAX(a) *max_element(ALL(a))

using ll = long long;
using db = long double;
using i128 = __int128_t;
using u32 = uint32_t;
using u64 = uint64_t;

const int INF=INT_MAX/2;
const ll LINF=LLONG_MAX/4;
const db DINF=numeric_limits<db>::infinity();
const int MOD=998244353;
const int MOD2=1000000007;
const db EPS=1e-9;
const db PI=acos(db(-1));

template<class T>
using PQ = priority_queue<T,vector<T>,greater<T>>;

template<class T,class U>
bool chmin(T &a,U b){return b<a?a=b,1:0;}
template<class T,class U>
bool chmax(T &a,U b){return a<b?a=b,1:0;}
template<class T,class U>
T SUM(const U &a){return accumulate(ALL(a),T{});}

template<class T>
using ordered_set = tree<T,null_type,less<T>,rb_tree_tag,
    tree_order_statistics_node_update>;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());
mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch()
    .count());
```

.bashrc 3 lines

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \
  -fsanitize=undefined,address'
xmodmap -e 'clear lock' -e 'keycode 66=less greater' #caps = <>
```

hash.sh 3 lines

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

stress.sh 11 lines

```
# Stress test: gen takes a seed (mt19937 rng(atoi(argv[1]))),
# sol and brute read stdin. Stops at the first difference.
for((i=1;;i++)); do
    ./gen $i > in
    ./sol < in > out1
    ./brute < in > out2
    if ! cmp -s out1 out2; then
        echo "WA on seed $i"; cat in; break
    fi
    [ $((i%100)) -eq 0 ] && echo "$i ok"
done
```

Mathematics (2)

2.1 Equations

$$ax^2+bx+c=0 \Rightarrow x=\frac{-b\pm\sqrt{b^2-4ac}}{2a}$$

The extremum is given by $x=-b/2a$.

$$\begin{aligned} ax+by&=e \\ cx+dy&=f \end{aligned} \Rightarrow \begin{aligned} x&=\frac{ed-bf}{ad-bc} \\ y&=\frac{af-ec}{ad-bc} \end{aligned}$$

In general, given an equation $Ax=b$, the solution to a variable x_i is given by

$$x_i=\frac{\det A_i'}{\det A}$$

where A_i' is A with the i'th column replaced by b.

2.2 Recurrences

If $a_n=c_1a_{n-1}+\cdots+c_ka_{n-k}$, and $r_1,\ldots,r_k$ are distinct roots of $x^k-c_1x^{k-1}-\cdots-c_k$, there are $d_1,\ldots,d_k$ s.t.

$$a_n=d_1r_1^n+\cdots+d_kr_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n=(d_1n+d_2)r^n$.

2.3 Trigonometry

$$\sin(v+w)=\sin v\cos w+\cos v\sin w$$
$$\cos(v+w)=\cos v\cos w-\sin v\sin w$$

$$\tan(v+w)=\frac{\tan v+\tan w}{1-\tan v\tan w}$$
$$\sin v+\sin w=2\sin\frac{v+w}{2}\cos\frac{v-w}{2}$$
$$\cos v+\cos w=2\cos\frac{v+w}{2}\cos\frac{v-w}{2}$$
$$(V+W)\tan(v-w)/2=(V-W)\tan(v+w)/2$$

where V,W are lengths of sides opposite angles v,w.

$$a\cos x+b\sin x=r\cos(x-\phi)$$
$$a\sin x+b\cos x=r\sin(x+\phi)$$

where $r=\sqrt{a^2+b^2},\phi=\text{atan2}(b,a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a,b,c

Semiperimeter: $p=\frac{a+b+c}{2}$

Area: $A=\sqrt{p(p-a)(p-b)(p-c)}$

Circumradius: $R=\frac{abc}{4A}$

Inradius: $r=\frac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a=\frac{1}{2}\sqrt{2b^2+2c^2-a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b+c}\right)^2\right]}$$

Law of sines: $\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

Law of tangents: $\frac{a+b}{a-b} = \frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

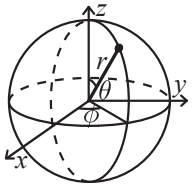
2.4.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin\theta = F\tan\theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.4.3 Spherical coordinates



$$\begin{array}{cc} x = r\sin\theta\cos\phi & r = \sqrt{x^2+y^2+z^2} \\ y = r\sin\theta\sin\phi & \theta = \operatorname{acos}(z/\sqrt{x^2+y^2+z^2}) \\ z = r\cos\theta & \phi = \operatorname{atan2}(y,x) \end{array}$$

2.5 Derivatives/Integrals

$$\begin{array}{cc} \frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} & \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}} \\ \frac{d}{dx}\tan x = 1 + \tan^2 x & \frac{d}{dx}\arctan x = \frac{1}{1+x^2} \\ \int \tan ax = -\frac{\ln|\cos ax|}{a} & \int x\sin ax = \frac{\sin ax - ax\cos ax}{a^2} \\ \int e^{-x^2} = \frac{\sqrt{\pi}}{2}\operatorname{erf}(x) & \int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1) \end{array}$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.7 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x. It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x xp_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.8.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\mathrm{Bin}(n,p)$, $n = 1, 2, \ldots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k}p^k(1-p)^{n-k}$$

$$\mu = np,\ \sigma^2 = np(1-p)$$

$\mathrm{Bin}(n,p)$ is approximately $\mathrm{Po}(np)$ for small p.

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\mathrm{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1},\ k = 1, 2, \ldots$$

$$\mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\mathrm{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda}\frac{\lambda^k}{k!}, k = 0, 1, 2, \ldots$$

$$\mu = \lambda,\ \sigma^2 = \lambda$$

2.8.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\mathrm{U}(a,b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2},\ \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\mathrm{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda},\ \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ are independent then

$$aX_1 + bX_2 + c \sim \mathcal{N}(a\mu_1 + b\mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \ldots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi\mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i. π_j/π_i is the expected number of visits in state j between two visits in state i.

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k\to\infty} \mathbf{P}^k = \mathbf{1}\pi$.

A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j, is $a_{ij} = p_{ij} + \sum_{k\in\mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i, is $t_i = 1 + \sum_{k\in\mathbf{G}} p_{ki} t_k$.

Data structures (3)

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided). d77092, 7 lines

```
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll,int,chash> h({},{},{},{},{1<<16});
```

line-container.hpp

Description: Line Container (Minimize).
Time: $\mathcal{O}(\log N)$ 24a5c4, 37 lines

```
struct Line{
  static bool querymode;
    ll m,c;
    mutable ll p;
    Line(ll m,ll c):m(m),c(c),p(0){}
    Line(ll p):m(0),c(0),p(p){}
    bool operator<(const Line &o)const{
        return querymode?p<o.p:m>o.m;
    }
};

bool Line::querymode=false;

struct LineContainer:multiset<Line>{
    ll div(ll a,ll b){
        return a/b-((a^b)<0&&a%b);
    }
    bool isect(iterator x,iterator y){
        if(y==end())return x->p=LINF,false;
        if(x->m==y->m)x->p=x->c<=y->c?LINF:-LINF;
        else x->p=div(x->c-y->c,y->m-x->m);
        return x->p>=y->p;
    }
    void add(ll m,ll c){
        auto x=insert(Line(m,c)),y=next(x);
        while(isect(x,y))y=erase(y);
        if((y=x)!=begin()&&isect(--x,y))isect(x,erase(y));
        while((y=x)!=begin()&&(--x)->p>=y->p)isect(x,erase(y));
    }
    ll get(ll x){
        if(empty())return LINF;
        Line::querymode=true;
        auto l=lower_bound(Line(x));
        Line::querymode=false;
        return l->m*x+l->c;
    }
};
```

treap.hpp

Description: Treap (1-indexed, 0 = null) with lazy reverse and lazy add (T needs +, T*int, ==). Weight-0 nodes take a position but no value. root/pos use parent pointers (not with P or pending rev). N about $2\ln n$ per op if P. 2e8d52, 64 lines

```
template<int N,class T,bool P=false>
struct Treap{
    struct Node{int l,r,p,sz,w,c;bool rev;T val,sum,lz;} t[N];
    int cnt=0;
    int make(T v,int w=1){
        t[++cnt]={0,0,0,1,w,w,0,v,w?v:T(),T()};return cnt;}
    int cp(int u){ // clone before writing (persistent only)
        if(!P||!u)return u;
        t[++cnt]=t[u];
        return cnt;
    }
    void pull(int u){
        auto &[l,r,p,sz,w,c,rev,val,sum,lz]=t[u];
        sz=t[l].sz+1+t[r].sz,c=t[l].c+w+t[r].c;
        sum=t[l].sum+t[r].sum;
        if(w)sum=sum+val;
        t[l].p=t[r].p=u;
    }
    void flip(int &u){
        if(!u)return;
        u=cp(u),swap(t[u].l,t[u].r),t[u].rev^=1;
    }
    void apply(int &u,T x){ // add x to every value in u
        if(!u)return;
        u=cp(u);
        auto &a=t[u];
        a.val=a.val+x,a.sum=a.sum+x*a.c,a.lz=a.lz+x;
    }
    void push(int u){
        if(t[u].rev)flip(t[u].l),flip(t[u].r),t[u].rev=0;
        T &x=t[u].lz;
        if(!(x==T()))apply(t[u].l,x),apply(t[u].r,x),x=T();
    }
    pair<int,int> split(int u,int k){ // first k elements, rest
        if(!u)return {0,0};
        u=cp(u),push(u);
        if(t[t[u].l].sz>=k){
            auto [a,b]=split(t[u].l,k);
            t[u].l=b,pull(u),t[u].p=0;
            return {a,u};
        }
        auto [a,b]=split(t[u].r,k-t[t[u].l].sz-1);
        t[u].r=a,pull(u),t[u].p=0;
        return {u,b};
    }
    int merge(int a,int b){
        if(!a||!b)return t[a^b].p=0,a^b;
        if(int(rng()%(t[a].sz+t[b].sz))<t[a].sz){
            a=cp(a),push(a);
            t[a].r=merge(t[a].r,b),pull(a),t[a].p=0;
            return a;
        }
        b=cp(b),push(b);
        t[b].l=merge(a,t[b].l),pull(b),t[b].p=0;
        return b;
    }
    int root(int u){while(t[u].p)u=t[u].p;return u;}
    int pos(int u){ // index of node u in its sequence
        int k=t[t[u].l].sz;
        for(;t[u].p;u=t[u].p)
            if(t[t[u].p].r==u)k+=t[t[t[u].p].l].sz+1;
        return k;
    } // (0-indexed)
};
```

segment-tree-beats.hpp

Description: Segment Tree Beats. Supports range chmin, range chmax, range add and range sum on 0-indexed inclusive $[l, r]$. Construct with SegmentTreeBeats(n,f) where f(i) gives a[i].
Time: amortized $\mathcal{O}\left(\log^2 N\right)$ per operation (with range add). e3b34a, 84 lines

```
struct SegmentTreeBeats{
    struct Node{
        ll sum=0,add=0,mn=LINF,mn2=LINF,fn=0;
        ll mx=-LINF,mx2=-LINF,fx=0;
        Node(){}
        Node(ll v):sum(v),mn(v),fn(1),mx(v),fx(1){}
        friend Node operator+(const Node &l,const Node &r){
            Node s;s.sum=l.sum+r.sum;
            auto &a=l.mx<r.mx?r:l,&b=l.mx<r.mx?l:r;
            s.mx=a.mx,s.fx=a.fx+(a.mx==b.mx?b.fx:0);
            s.mx2=max(a.mx2,a.mx==b.mx?b.mx2:b.mx);
            auto &c=r.mn<l.mn?r:l,&d=r.mn<l.mn?l:r;
            s.mn=c.mn,s.fn=c.fn+(c.mn==d.mn?d.fn:0);
            s.mn2=min(c.mn2,c.mn==d.mn?d.mn2:d.mn);
            return s;
        }
        void apply(ll k,ll v){
            sum+=k*v,mx+=v,mx2+=v,mn+=v,mn2+=v,add+=v;
        }
        void chmin(ll v){
            if(v<mx)sum+=(v-mx)*fx,mn=mn==mx?v:mn,
                mn2=mn2==mx?v:mn2,mx=v;
        }
        void chmax(ll v){
            if(v>mn)sum+=(v-mn)*fn,mx=mx==mn?v:mx,
                mx2=mx2==mn?v:mx2,mn=v;
        }
    };
    int n;vector<Node> t;
    SegmentTreeBeats(){}
    SegmentTreeBeats(int n){init(n,[&](int){return 0;});}
    template<class F>
    SegmentTreeBeats(int n,const F &f){init(n,f);}
```

```
    template<class F>
    void init(int _n,const F &f){
        n=_n;int s=1;while(s<n*2)s<<=1;
        t.assign(s,Node());build(f);
    }
    template<class F>
    void build(int l,int r,int i,const F &f){
        if(l==r)return void(t[i]=f(l));
        int m=(l+r)/2;
        build(l,m,i*2,f),build(m+1,r,i*2+1,f),pull(i);
    }
    void pull(int i){t[i]=t[i*2]+t[i*2+1];}
    void push(int l,int r,int i){
        int m=(l+r)/2;Node &p=t[i],&a=t[i*2],&b=t[i*2+1];
        a.apply(m-l+1,p.add),a.chmin(p.mx),a.chmax(p.mn);
        b.apply(r-m,p.add),b.chmin(p.mx),b.chmax(p.mn);
        p.add=0;
    }
    void range_add(int l,int r,int i,int x,int y,ll v){
        if(y<l||r<x)return;
        if(x<=l&&r<=y)return t[i].apply(r-l+1,v);
        int m=(l+r)/2;push(l,r,i),range_add(l,m,i*2,x,y,v);
        range_add(m+1,r,i*2+1,x,y,v),pull(i);
    }
    void range_chmin(int l,int r,int i,int x,int y,ll v){
        if(y<l||r<x||t[i].mx<=v)return;
        if(x<=l&&r<=y&&t[i].mx2<v)return t[i].chmin(v);
        int m=(l+r)/2;push(l,r,i),range_chmin(l,m,i*2,x,y,v);
        range_chmin(m+1,r,i*2+1,x,y,v),pull(i);
    }
    void range_chmax(int l,int r,int i,int x,int y,ll v){
        if(y<l||r<x||t[i].mn>=v)return;
        if(x<=l&&r<=y&&t[i].mn2>v)return t[i].chmax(v);
        int m=(l+r)/2;push(l,r,i),range_chmax(l,m,i*2,x,y,v);
        range_chmax(m+1,r,i*2+1,x,y,v),pull(i);
    }
    ll query(int l,int r,int i,int x,int y){
        if(y<l||r<x)return 0;
        if(x<=l&&r<=y)return t[i].sum;
        int m=(l+r)/2;push(l,r,i);
        return query(l,m,i*2,x,y)+query(m+1,r,i*2+1,x,y);
    }
    template<class F>
    void build(const F &f){if(n)build(0,n-1,1,f);}
    void range_add(int x,int y,ll v){range_add(0,n-1,1,x,y,v);}
    void range_chmin(int x,int y,ll v){
        range_chmin(0,n-1,1,x,y,v);}
    void range_chmax(int x,int y,ll v){
        range_chmax(0,n-1,1,x,y,v);}
    ll query(int x,int y){return query(0,n-1,1,x,y);}
};
```

Number Theory (4)

euclid.hpp

Description: Finds two integers x and y,such that $ax+by=\gcd(a,b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod b$. $x=x_0+k*(b/g)$ $y=y_0-k*(a/g)$

"src/number-theory/euclid.hpp" 33ba8f, 5 lines

```
ll euclid(ll a,ll b,ll &x,ll &y){
  if(!b)return x=1,y=0,a;
  ll d=euclid(b,a%b,y,x);
  return y-=a/b*x,d;
}
```

crt.hpp

Description: Chinese Remainder Theorem.
crt(a, m, b, n) computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \text{lcm}(m,n)$. Assumes $mn < 2^{62}$. If x0 and y0 is one of the solutions of ax + by = g, then the general solution is x = x0 + k * (b / g) and y = y0 - k * (a / g).
Time: $\log(n)$

"src/number-theory/euclid.hpp" cfd447, 7 lines

```
ll crt(ll a,ll m,ll b,ll n){
  if(n>m)swap(a,b),swap(m,n);
  ll x,y,g=euclid(m,n,x,y);
  if((a-b)%g!=0)return -1LL; // no solution
  x=(b-a)%n*x%n/g*m+a;
  return x<0?x+m*n/g:x;
}
```

floor-sum.hpp

Description: Floor sum function. $f(a,b,c,n)=\sum_{x=0}^{n}\lfloor\frac{ax+b}{c}\rfloor$ becareful when a,b,c are negetive (use custom floor division and mod instead)
Time: $\mathcal{O}(\log a)$

d088d2, 7 lines

```
ll floor_sum(ll a,ll b,ll c,ll n){
    ll res=n*(n+1)/2*(a/c)+(n+1)*(b/c);
    a%=c,b%=c;
    if(a==0)return res;
    ll m=(a*n+b)/c;
    return res+n*m-floor_sum(c,c-b-1,a,m-1);
}
```

barrett.hpp

Description: Barrett reduction, used for fast division / modulo.

3b7737, 29 lines

```
struct Barrett{
    using u32 = uint32_t;
    using u64 = uint64_t;
    using u128 = __uint128_t;
    u32 m;
    u64 im;
    explicit Barrett(u32 m=1):m(m),im((u64)(-1)/m+1){}
    u32 umod()const{return m;}
    u32 modulo(u64 z){
        if(m==1)return 0;
        u64 x=(u64)(((u128)(z)*im)>>64);
        u64 y=x*m;
        return (z-y+(z<y?m:0));
    }
    u64 floor(u64 z){
        if(m==1)return z;
        u64 x=(u64)(((u128)(z)*im)>>64);
        u64 y=x*m;
        return (z<y?x-1:x);
    }
    pair<u64,u32> divmod(u64 z){
        if (m==1)return {z,0};
        u64 x=(u64)(((u128)(z)*im)>>64);
        u64 y=x*m;
        if (z<y)return {x-1,z-y+m};
        return {x,z-y};
    }
    u32 mul(u32 a,u32 b){return modulo(u64(a)*b);}
};
```

primitive-root.hpp

Description: Smallest primitive root of a prime p.
Time: $\mathcal{O}(Ans \log\phi(n)\log n)$

659395, 29 lines

```
int modpow(int a,int b,int mod){
    int res=1;
    while(b>0){
        if(b&1)res=1LL*res*a%mod;
        a=1LL*a*a%mod;
        b>>=1;
    }
    return res;
}

int primitive_root(int p){
    vector<int> fact;
    int phi=p-1,n=phi;
    for(int i=2; i*i<=n;i++){
        if(n%i==0){
            fact.emplace_back(i);
            while(n%i==0)n/=i;
        }
    }
    if(n>1)fact.emplace_back(n);
    for(int res=1;res<p;res++){
        bool ok=true;
        for(int i=0;i<fact.size()&&ok;i++){
            ok&=(modpow(res,phi/fact[i],p)!=1);
        }
        if(ok)return res;
    }
    return -1;
}
```

factorization.hpp

Description: Deterministic Miller-Rabin primality test for all 64-bit n and Pollard rho factorization. factor(n) returns the prime factors of n sorted, with multiplicity (empty for $n=1$).
Usage: factor(360); // {2,2,2,3,3,5}
Time: $\mathcal{O}\left(n^{1/4}\right)$ gcd/mulmod for factor, $\mathcal{O}(7\log n)$ for is_prime.

364a86, 35 lines

```
u64 mulmod(u64 a,u64 b,u64 m){return (unsigned __int128)a*b%m;}
u64 powmod(u64 b,u64 e,u64 m){
    u64 r=1;
    for(;e;b=mulmod(b,b,m),e>>=1)if(e&1)r=mulmod(r,b,m);
    return r;
}
bool is_prime(u64 n){
    if(n<2||n%6%4!=1)return (n|1)==3;
    u64 s=__builtin_ctzll(n-1),d=n>>s;
    for(u64 a:{2,325,9375,28178,450775,9780504,1795265022}){
        u64 p=powmod(a%n,d,n),i=s;
        while(p!=1&&p!=n-1&&a%n&&i--)p=mulmod(p,p,n);
        if(p!=n-1&&i!=s)return 0;
    }
    return 1;
}
u64 pollard(u64 n){
    u64 x=0,y=0,t=30,prd=2,i=1,q;
    auto f=[&](u64 x){return mulmod(x,x,n)+i;};
    while(t++%40||__gcd(prd,n)==1){
        if(x==y)x=++i,y=f(x);
        if((q=mulmod(prd,max(x,y)-min(x,y),n)))prd=q;
        x=f(x),y=f(f(y));
    }
    return __gcd(prd,n);
}
vector<u64> factor(u64 n){
    if(n==1)return {};
    if(is_prime(n))return {n};
    u64 x=n%2?pollard(n):2;
    auto l=factor(x),r=factor(n/x);
    l.insert(l.end(),ALL(r));
    sort(ALL(l));
    return l;
}
```

discrete-log.hpp

Description: Baby-step giant-step. Returns the smallest $x \geq 0$ such that $a^x \equiv b \pmod m$, or -1 if none. Works for any $m \geq 1$ (non-coprime a, m allowed). Assumes m^2 fits in `ll`.
Usage: `discrete_log(2,3,5); // 3`
Time: $\mathcal{O}\left(\sqrt{m}\right)$ 306d9a, 18 lines

```
ll discrete_log(ll a,ll b,ll m){
    a%=m,b%=m;
    ll k=1,add=0,g;
    while((g=gcd(a,m))>1){
        if(b==k)return add;
        if(b%g)return -1;
        b/=g,m/=g,add++,k=k*a/g%m;
    }
    ll n=sqrtl(m)+1,an=1;
    for(int i=0;i<n;i++)an=an*a%m;
    unordered_map<ll,ll> vals;
    for(ll q=0,cur=b;q<=n;q++)vals[cur]=q,cur=cur*a%m;
    for(ll p=1,cur=k;p<=n;p++){
        cur=cur*an%m;
        if(vals.count(cur))return n*p-vals[cur]+add;
    }
    return -1;
}
```

mod-sqrt.hpp

Description: Tonelli-Shanks. Returns x with $x^2 \equiv a \pmod p$ for prime p (the other root is $p - x$), or -1 if a is not a quadratic residue.
Usage: `mod_sqrt(2,7); // 3 or 4`
Time: $\mathcal{O}\left(\log^2 p\right)$ c5359b, 23 lines

```
ll mod_sqrt(ll a,ll p){
    auto mul=[&](ll x,ll y){return ll(i128(x)*y%p);};
    auto pw=[&](ll b,ll e){
        ll r=1;
        for(;e;b=mul(b,b),e>>=1)if(e&1)r=mul(r,b);
        return r;
    };
    a%=p;
    if(a<0)a+=p;
    if(a==0||p==2)return a;
    if(pw(a,(p-1)/2)!=1)return -1;
    ll s=p-1,r=0,z=2;
    while(s%2==0)s/=2,r++;
    while(pw(z,(p-1)/2)!=p-1)z++;
    ll x=pw(a,(s+1)/2),b=pw(a,s),g=pw(z,s);
    while(b!=1){
        ll t=b,m=0;
        while(t!=1)t=mul(t,t),m++;
        ll gs=pw(g,1LL<<(r-m-1));
        g=mul(gs,gs),x=mul(x,gs),b=mul(b,g),r=m;
    }
    return x;
}
```

prime-counting.hpp

Description: Lucy_Hedgehog sieve. For a completely multiplicative f and every $v = \lfloor n/i \rfloor$ computes $G(v) = \sum_{p \leq v,\, p \text{ prime}} f(p)$. `pre(v)` must return $\sum_{x=2}^{v} f(x)$ as `T` (e.g. $v - 1$ for π, $v(v+1)/2 - 1$ for sum of primes). Read with `g(v)` for $v = \lfloor n/i \rfloor$ (any $v \leq \sqrt{n}$ works). Use `T=ll` for π, `i128` for $\sum p$ ($n \leq 10^{12}$) or a modint. For a polynomial $f(p) = \sum c_k p^k$ run once per k and combine.
Usage: `Lucy<ll> pi(n,[](ll v){return v-1;}); pi(n/3);`
`Lucy<i128> sp(n,[](ll v){return (i128)v*(v+1)/2-1;});`
Time: $\mathcal{O}\left(n^{3/4}/\log n\right)$, $\pi(10^{11})$ in ≈ 0.2s. ab6fbd, 19 lines

```
template<class T>
struct Lucy{
    ll n;int s;vector<T> lo,hi; // lo[v]=G(v), hi[i]=G(n/i)
    template<class F>
    Lucy(ll n,F pre):n(n),s(sqrtl(n)),lo(s+1),hi(s+1){
        for(int i=1;i<=s;i++)lo[i]=pre(i),hi[i]=pre(n/i);
        for(int p=2;p<=s;p++){
            if(lo[p]==lo[p-1])continue;
            T fp=lo[p]-lo[p-1],b=lo[p-1];
            ll q=(ll)p*p;int e=min<ll>(s,n/q);
            for(int i=1;i<=e;i++){
                ll d=(ll)i*p;
                hi[i]-=fp*((d<=s?hi[d]:lo[n/d])-b);
            }
            for(int v=s;v>=q;v--)lo[v]-=fp*(lo[v/p]-b);
        }
    }
    T operator()(ll v){return v<=s?lo[v]:hi[n/v];}
};
```

min25-sieve.hpp

Description: min_25 sieve: $\sum_{x=1}^{n} f(x)$ for multiplicative f. Supply `g(v)`$= \sum_{p \leq v} f(p)$ for every $v = \lfloor n/i \rfloor$ (combine `Lucy` sums of p^k when $f(p)$ is a polynomial) and `fpe(p,e)`$= f(p^e)$ as `T`. Assumes $f(1) = 1$. `T`: `ll`/`i128`/modint (watch overflow).
Usage: `Lucy<mint> c(n,[](ll v){return mint(v-1);});`
`Lucy<mint> s(n,[](ll v){return mint(v)*(v+1)/2-1;});`
`auto g=[&](ll v){return s(v)-c(v);}; // phi(p)=p-1`
`auto fpe=[](ll p,int e){mint r=p-1;while(--e)r*=p;return r;};`
`mint ans=min25<mint>(n,g,fpe);`
Time: $\mathcal{O}\left(n^{3/4}/\log n\right)$, $n = 10^{10}$ in ≈ 0.3s.
"src/number-theory/prime-counting.hpp" 8db825, 19 lines

```
template<class T,class G,class F>
T min25(ll n,G g,F fpe){
    int s=sqrtl(n);vector<int> pr;vector<bool> c(s+1);
    for(int i=2;i<=s;i++)if(!c[i]){
        pr.pb(i);
        for(ll j=(ll)i*i;j<=s;j+=i)c[j]=1;
    }
    // S(m,j) = sum of f(x), 2<=x<=m, lpf(x)>=pr[j]
    function<T(ll,int)> S=[&](ll m,int j){
        T r=g(m)-(j?g(pr[j-1]):T(0));
        for(int i=j;i<SZ(pr)&&(ll)pr[i]*pr[i]<=m;i++){
            ll p=pr[i],q=p;
            for(int e=1;q*p<=m;e++,q*=p)
                r+=fpe(p,e)*S(m/q,i+1)+fpe(p,e+1);
        }
        return r;
    };
    return S(n,0)+T(1);
}
```

4.1 Multiplicative Functions

4.1.1 Definitions

f multiplicative: $f(1) = 1$, $f(ab) = f(a)f(b)$ for $\gcd(a, b) = 1$, so $f(n) = \prod f(p^e)$. Completely multiplicative: for all a, b.

f	$f(p^e)$
$\mathbf{1}(n) = 1$	1
$\mathrm{id}_k(n) = n^k$	p^{ek}
$\varepsilon(n) = [n = 1]$	0
μ	-1 if $e = 1$, else 0
φ	$p^{e-1}(p - 1)$
$d = \sigma_0$	$e + 1$
$\sigma_k = \sum_{d \mid n} d^k$	$\frac{p^{k(e+1)}-1}{p^k-1}$
$\lambda = (-1)^\Omega$	$(-1)^e$

$\omega(n) = \#$distinct primes, $\Omega(n) = \#$primes with mult. (additive, not mult.); 2^ω, λ are mult. $\varphi(n) = n \prod_{p \mid n} (1 - \frac{1}{p})$,
$\sum_{i \leq n, \gcd(i,n)=1} i = \frac{n\varphi(n)+[n=1]}{2}$.

4.1.2 Dirichlet convolution

$(f * g)(n) = \sum_{d \mid n} f(d) g(\frac{n}{d})$. Commutative, associative, identity ε; f, g mult. $\Rightarrow f * g$ mult. f^{-1} exists iff $f(1) \neq 0$; for compl. mult. f: $f^{-1} = \mu f$. Computing $f * g$ for all $n \leq N$: $O(N \log N)$ (loop d, multiples).

$$\mu * \mathbf{1} = \varepsilon \qquad \varphi * \mathbf{1} = \mathrm{id}$$
$$\mathbf{1} * \mathbf{1} = d \qquad \mathrm{id}_k * \mathbf{1} = \sigma_k$$
$$\varphi = \mu * \mathrm{id} \qquad \lambda * \mathbf{1} = [n \text{ is perfect square}]$$
$$\mu^2 * \mathbf{1} = 2^\omega \qquad \varphi * d = \sigma_1$$
$$|\mu| * \lambda = \varepsilon \qquad \mu * \mathrm{id}_k = J_k$$

(J_k Jordan totient, $J_1 = \varphi$.) $d(n^2) = \sum_{d \mid n} 2^{\omega(d)}$,
$\sigma_0(ij) = \sum_{x \mid i, y \mid j} [\gcd(x, y) = 1]$.

4.1.3 Möbius inversion

$$g = f * \mathbf{1} \iff f = g * \mu$$

$$g(n) = \sum_{n \mid m \leq N} f(m) \iff f(n) = \sum_{n \mid m \leq N} \mu(\tfrac{m}{n}) g(m)$$

Use: $[\gcd = 1] = \sum_{d \mid \gcd} \mu(d)$. $g(d)$=#tuples with $d \mid \gcd$ (easy), $f(d)$=#tuples with $\gcd = d$: $f(d) = g(d) - \sum_{k \geq 2} f(kd)$, d descending.

$$\sum_{i \leq n} \sum_{j \leq m} [\gcd(i, j) = 1] = \sum_d \mu(d) \lfloor \tfrac{n}{d} \rfloor \lfloor \tfrac{m}{d} \rfloor$$

$$\sum_{i,j \leq n} \gcd(i, j) = \sum_d \varphi(d) \lfloor \tfrac{n}{d} \rfloor^2$$

$$\sum_{i=1}^{n} \gcd(i, n) = \sum_{d \mid n} d\, \varphi(\tfrac{n}{d})$$

$$\sum_{i,j \leq n} [\gcd(i, j) = 1] = 2\Phi(n) - 1, \quad \Phi(n) = \sum_{i \leq n} \varphi(i)$$

$$\sum_{i \leq n} \mathrm{lcm}(i, n) = \tfrac{n}{2} \left(1 + \sum_{d \mid n} d\, \varphi(d)\right)$$

4.1.4 Floor blocks

$\lfloor n/i \rfloor$ takes $\leq 2\sqrt{n}$ values; $\lfloor \lfloor n/a \rfloor / b \rfloor = \lfloor n/ab \rfloor$. Block with value $\lfloor n/i \rfloor$ is $[i, \lfloor n/\lfloor n/i \rfloor \rfloor]$: `for(l=1;l<=n;l=r+1){r=n/(n/l); ...}`. Store values at $v = \lfloor n/i \rfloor$ in two arrays: `lo[v]` ($v \leq \sqrt{n}$), `hi[n/v]`.

4.1.5 Prefix sums

$S_f(n) = \sum_{i \le n} f(i)$. Hyperbola, $h = f * g$, $m = \lfloor \sqrt{n} \rfloor$:

$$S_h(n) = \sum_{d \le m} \left(f(d) S_g(\lfloor \tfrac{n}{d} \rfloor) + g(d) S_f(\lfloor \tfrac{n}{d} \rfloor) \right) - S_f(m) S_g(m)$$

Simplest: $S_h(n) = \sum_{d \le n} f(d) S_g(\lfloor n/d \rfloor)$ over blocks.

Du's sieve: pick g with S_g, S_{f*g} in $O(1)$:

$$g(1) S_f(n) = S_{f*g}(n) - \sum_{i=2}^{n} g(i)\, S_f(\lfloor \tfrac{n}{i} \rfloor)$$

Precompute S_f up to $n^{2/3}$ (linear sieve), memo rest (`hi[n/v]`): $O(n^{2/3})$. φ: $g = \mathbf{1}$, $S_{f*g} = \frac{n(n+1)}{2}$. μ: $g = \mathbf{1}$, $S_{f*g} = 1$. $\varphi \cdot \mathrm{id}$: $g = \mathrm{id}$, $f * g = \mathrm{id}_{k+1}$.

Linear sieve for mult. f: keep `lp[i]` and `pw[i]` (power of lpf in i). For $i \cdot p$ with $p \le$ `lp[i]`: if $p <$ `lp[i]`, $f(ip) = f(i)f(p)$; else with $q =$ `pw[i]`: $f(ip) = f(\frac{i}{q})f(qp)$ (need $f(p^e)$ directly, or use $\varphi(ip) = \varphi(i)p$, $\mu(ip) = 0$).

4.1.6 Which method

- $n \le 10^7$: linear sieve.
- $\sum_{p \le n} p^k$, $\pi(n)$, $n \le 10^{11}$: Lucy `prime-counting`.
- $\sum f$, $f(p)$ poly in p, $f(p^e)$ easy: `min25`, $n \le 10^{11}$.
- $f * g$ and g easy to sum (e.g. φ, μ): Du, $n \le 10^{11}$.
- $\sum f(i) g(\lfloor n/i \rfloor)$: floor blocks + S_f at all $\lfloor n/i \rfloor$ (Lucy and memoized Du give all of them).

Combinatorial (5)

5.1 Permutations

5.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

5.1.2 Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n} \right)$$

5.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

5.1.4 Burnside's lemma

Given a group G of symmetries and a set X, the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts "configurations" (of some sort) of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n} \sum_{k \mid n} f(k) \phi(n/k).$$

5.1.5 Pólya's enumeration theorem

Let G be a group of permutations on a set of n elements. The number of non-equivalent colorings of the set using m colors is:

$$\frac{1}{|G|} \sum_{g \in G} m^{c(g)}$$

where $c(g)$ is the number of cycles in the cycle decomposition of the permutation g.

5.2 Partitions and subsets

5.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	∼2e5	∼2e8

5.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + ... + n_1 p + n_0$ and $m = m_k p^k + ... + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod{p}$.

5.2.3 The Twelvefold Way

Ways to put n items into m boxes:

Items	Boxes	Any	≤ 1 (inj)	≥ 1 (surj)
Dist	Dist	m^n	$P(m,n)$	$m!S(n,m)$
Indist	Dist	$\binom{n+m-1}{m-1}$	$\binom{m}{n}$	$\binom{n-1}{m-1}$
Dist	Indist	$\sum_{k=1}^{m} S(n,k)$	$[n \le m]$	$S(n,m)$
Indist	Indist	$\sum_{k=1}^{m} p(n,k)$	$[n \le m]$	$p(n,m)$

$S(n,k)$: Stirling 2nd kind, $p(n,k)$: partitions of n into exactly k parts, $p(n,k) = p(n-1,k-1) + p(n-k,k)$.

5.2.4 Hook Length Formula

Number of Standard Young Tableaux of a partition shape $\lambda \vdash n$:

$$f^\lambda = \frac{n!}{\prod_{(i,j) \in \lambda} h_\lambda(i,j)}$$

where the hook length $h_\lambda(i,j)$ is the number of cells directly to the right and directly below cell (i,j), including itself.

5.3 General purpose numbers

5.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).
$B[0, \ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} i^m = \frac{1}{m+1} \sum_{k=0}^{m} \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^\infty f(x)dx - \sum_{k=1}^{\infty} \frac{B_k}{k!} f^{(k-1)}(m)$$

$$\approx \int_m^\infty f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

5.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$

$$\textstyle\sum_{k=0}^{n} c(n,k) x^k = x(x+1)\ldots(x+n-1)$$

$c(8,k) = 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

5.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \ge j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k} (-1)^j \binom{n+1}{j} (k+1-j)^n$$

5.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

5.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$$

5.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$
with degrees d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

Prüfer code: bijection between labeled trees on n vertices and sequences in $[n]^{n-2}$ (repeatedly remove the smallest leaf, write its neighbor). Vertex i appears exactly $d_i - 1$ times, so counting trees with degrees d_i is a multinomial:

$$\binom{n-2}{d_1-1,\ldots,d_n-1} = \frac{(n-2)!}{\prod(d_i-1)!}$$

Joining k components of sizes s_i by $k-1$ edges such that component i gets d_i of the new edges (treat each component as one Prüfer symbol, each edge end picks one of s_i vertices):

$$\frac{(k-2)!}{\prod(d_i-1)!}\prod s_i^{d_i}$$

Summing over all d_i gives $s_1 \cdots s_k\, n^{k-2}$.

5.3.7 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \ldots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

5.3.8 Binomial Identities

- **Vandermonde's Identity:** $\sum_k \binom{r}{k}\binom{s}{n-k} = \binom{r+s}{n}$
- **Hockey-stick Identity:** $\sum_{i=r}^{n}\binom{i}{r} = \binom{n+1}{r+1}$
- **Row sum of squares:** $\sum_{k=0}^{n}\binom{n}{k}^2 = \binom{2n}{n}$

5.3.9 Min-Max Identity

For any finite set of values S:

$$\max(S) = \sum_{\emptyset \neq T \subseteq S}(-1)^{|T|-1}\min(T)$$

$$\min(S) = \sum_{\emptyset \neq T \subseteq S}(-1)^{|T|-1}\max(T)$$

Also holds perfectly for expected values:
$\mathbb{E}[\max(S)] = \sum(-1)^{|T|-1}\mathbb{E}[\min(T)]$.

5.3.10 Lagrange Inversion Formula

If $A(x) = x\phi(A(x))$ where $\phi(0) \neq 0$, then the coefficients of $A(x)$ are given by:

$$[x^n]A(x) = \frac{1}{n}[u^{n-1}]\big(\phi(u)\big)^n$$

More generally, for any function $H(x)$:
$[x^n]H(A(x)) = \frac{1}{n}[u^{n-1}]H'(u)\big(\phi(u)\big)^n$.

Polynomials (6)

6.1 Generating Functions

6.1.1 Common Series

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \cdots = \sum_{n=0}^{\infty} x^n$$

$$-\ln(1-x) = x + \frac{x^2}{2} + \frac{x^3}{3} + \cdots = \sum_{n=1}^{\infty}\frac{x^n}{n}$$

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots = \sum_{n=0}^{\infty}\frac{x^n}{n!}$$

$$(1-x)^{-k} = \sum_{n=0}^{\infty}\binom{n+k-1}{k-1}x^n$$

6.1.2 Famous Polynomials

Bell numbers B_n are the number of partitions of n distinct objects. let $B(x)$ be EGF of Bell numbers, then

$$B(x) = e^{e^x - 1} = \sum_{n=0}^{\infty} B_n \frac{x^n}{n!}$$

6.2 Newton's Method

If $F(Q) = 0$, then $Q_{2n} \equiv Q_n - \frac{F(Q_n)}{F'(Q_n)} \pmod{x^{2n}}$. To get $Q \bmod x^N$: start from $Q_1 = Q(0)$ (solve $F(Q_1) \equiv 0 \pmod{x}$), then apply the step for $n = 1, 2, 4, \ldots$ until $n \geq N$. Step n only needs $P \bmod x^{2n}$; truncate Q_{2n} to $2n$ terms. Total $O(N \log N)$ since $T(N) = T(N/2) + O(N \log N)$.

Pick F with $F(Q) = 0$ such that F, F' (F' is w.r.t. Q) only use already-available ops, and $F'(Q)(0) \neq 0$:

Want	$F(Q)$	$F'(Q)$	$Q - F/F'$
P^{-1}	$Q^{-1} - P$	$-Q^{-2}$	$Q(2 - PQ)$
e^P	$\ln Q - P$	Q^{-1}	$Q(1 + P - \ln Q)$
$\sqrt{P}$	$Q^2 - P$	$2Q$	$\frac{1}{2}(Q + PQ^{-1})$

Not $PQ - 1$ for P^{-1}: $F' = P$ would need P^{-1} itself. Also works for functional equations, e.g. $Q = xe^Q$: $F = Q - xe^Q$, $F' = 1 - xe^Q$.

$$Q = P^{-1} : Q_{2n} \equiv Q_n \cdot (2 - P \cdot Q_n) \pmod{x^{2n}}$$

$$Q = \ln P = \int \frac{P'}{P}\mathrm{d}x$$

$$Q = e^P : Q_{2n} \equiv Q_n(1 + P - \ln Q_n) \pmod{x^{2n}}$$

$$Q = \sqrt{P} : Q_{2n} \equiv \frac{1}{2}(Q_n + P \cdot Q_n^{-1}) \pmod{x^{2n}}$$

$$Q = P^k = \alpha^k x^{kt} e^{k \ln T}; P = \alpha \cdot x^t \cdot T, T(0) = 1$$

Starting values and requirements:
P^{-1}: $Q_1 = P(0)^{-1}$, needs $P(0) \neq 0$.
$\ln P$: no iteration (one inverse), needs $P(0) = 1$.
e^P: $Q_1 = 1$, needs $P(0) = 0$.
$\sqrt{P}$: $Q_1 = \sqrt{P(0)}$ (modular sqrt), compute Q_n^{-1} to $2n$ terms; if $P(0) = 0$, factor out x^{2t} first.
P^k: find lowest nonzero term αx^t; answer is 0 if $kt \geq N$.

formal-power-series.hpp

Description: basic operations of formal power series

"src/polynomials/ntt.hpp" 716a21, 147 lines

```
template<class mint>
struct FormalPowerSeries:vector<mint>{
    using vector<mint>::vector;
    using FPS = FormalPowerSeries;

    FPS &operator+=(const FPS &r){
        if(r.size()>this->size())this->resize(r.size());
        for(int i=0;i<r.size();i++)(*this)[i]+=r[i];
        return *this;
    }
    FPS &operator+=(const mint &r){
        if(this->empty())this->resize(1);
        (*this)[0]+=r;
        return *this;
    }
    FPS &operator-=(const FPS &r){
        if(r.size()>this->size())this->resize(r.size());
        for(int i=0;i<r.size();i++)(*this)[i]-=r[i];
        return *this;
```

```
    }
    FPS &operator-=(const mint &r){
        if(this->empty())this->resize(1);
        (*this)[0]-=r;
        return *this;
    }
    FPS &operator*=(const FPS &r){
        auto res=NTT<mint>()(*this,r);
        return *this=FPS(res.begin(),res.end());
    }
    FPS &operator*=(const mint &r){
        for(auto &a:*this)a*=r;
        return *this;
    }
    FPS &operator/=(const FPS &r){ // r.back()!=0
        if(this->size()<r.size())return *this=FPS();
        int n=this->size()-r.size()+1;
        return *this=(rev().pre(n)*r.rev().inv(n)).pre(n).rev()
            ;
    }
    FPS &operator%=(const FPS &r){
        *this-=*this/r*r;
        return *this=pre(r.size()-1).shrink();
    }

    friend FPS operator+(FPS l,const FPS &r){return l+=r;}
    friend FPS operator+(FPS l,const mint &r){return l+=r;}
    friend FPS operator+(const mint &l,FPS r){return r+=l;}
    friend FPS operator-(FPS l,const FPS &r){return l-=r;}
    friend FPS operator-(FPS l,const mint &r){return l-=r;}
    friend FPS operator-(const mint &l,FPS r){return -(r-l);}
    friend FPS operator*(FPS l,const FPS &r){return l*=r;}
    friend FPS operator*(FPS l,const mint &r){return l*=r;}
    friend FPS operator*(const mint &l,FPS r){return r*=l;}
    friend FPS operator/(FPS l,const FPS &r){return l/=r;}
    friend FPS operator%(FPS l,const FPS &r){return l%=r;}

    FPS operator-()const{return (*this)*-1;}

    FPS rev()const{
        FPS res(*this);
        reverse(res.begin(),res.end());
        return res;
    }
    FPS pre(int sz)const{
        FPS res(this->begin(),this->begin()+min((int)this->size
            (),sz));
        if(res.size()<sz)res.resize(sz);
        return res;
    }
    FPS shrink()const{
        FPS res(*this);
        while(!res.empty()&&res.back()==mint{})res.pop_back();
        return res;
    }
    FPS operator>>(int sz)const{
        if(this->size()<=sz)return {};
        FPS res(*this);
        res.erase(res.begin(),res.begin()+sz);
        return res;
    }
    FPS operator<<(int sz)const{
        FPS res(*this);
        res.insert(res.begin(),sz,mint{});
        return res;
    }
    FPS diff()const{
        const int n=this->size();
        FPS res(max(0,n-1));
        for(int i=1;i<n;i++)res[i-1]=(*this)[i]*mint(i);
        return res;
    }
    FPS integral()const{
        const int n=this->size();
        FPS res(n+1);
        res[0]=0;
        if(n>0)res[1]=1;
        ll mod=mint::get_mod();
        for(int i=2;i<=n;i++)res[i]=(-res[mod%i])*(mod/i);
        for(int i=0;i<n;i++)res[i+1]*=(*this)[i];
        return res;
    }
    mint eval(const mint &x)const{
        mint res=0,w=1;
        for(auto &a:*this)res+=a*w,w*=x;
        return res;
    }

    FPS inv(int deg=-1)const{
        assert(!this->empty()&&(*this)[0]!=mint(0));
        if(deg==-1)deg=this->size();
        FPS res{mint(1)/(*this)[0]};
        for(int i=2;i>>1<deg;i<<=1){
            res=(res*(mint(2)-res*pre(i))).pre(i);
        }
        return res.pre(deg);
    }
    FPS log(int deg=-1)const{
        assert(!this->empty()&&(*this)[0]==mint(1));
        if(deg==-1)deg=this->size();
        return (pre(deg).diff()*inv(deg)).pre(deg-1).integral()
            ;
    }
    FPS exp(int deg=-1)const{
        assert(this->empty()||(*this)[0]==mint(0));
        if(deg==-1)deg=this->size();
        FPS res{mint(1)};
        for(int i=2;i>>1<deg;i<<=1){
            res=(res*(pre(i)-res.log(i)+mint(1))).pre(i);
        }
        return res.pre(deg);
    }
    FPS pow(ll k,int deg=-1)const{
        const int n=this->size();
        if(deg==-1)deg=n;
        if(k==0){
            FPS res(deg);
            if(deg)res[0]=mint(1);
            return res;
        }
        for(int i=0;i<n;i++){
            if(__int128_t(i)*k>=deg)return FPS(deg,mint(0));
            if((*this)[i]==mint(0))continue;
            mint rev=mint(1)/(*this)[i];
            FPS res=(((*this*rev)>>i).log(deg)*k).exp(deg);
            res=((res*binpow((*this)[i],k))<<(i*k)).pre(deg);
            return res;
        }
        return FPS(deg,mint(0));
    }
};
```

fft.hpp

Description: Fast Fourier Transform
Time: $\mathcal{O}(N \log N)$

7503cc, 71 lines

```
template<class T=ll,int mod=0>
struct FFT{
    using vt = vector<T>;
    using cd = complex<db>;
    using vc = vector<cd> ;

    static const bool INT=is_integral<T>::value;

    static void fft(vc &a){
      int n=a.size(),L=31-__builtin_clz(n);
      vc rt(n);
      rt[1]=1;
      for(int k=2;k<n;k*=2){
        cd z=polar(db(1),PI/k);
        for(int i=k;i<2*k;i++)rt[i]=i&1?rt[i/2]*z:rt[i/2];
      }
      vector<int> rev(n);
      for(int i=1;i<n;i++)rev[i]=(rev[i/2]|(i&1)<<L)/2;
      for(int i=1;i<n;i++)if(i<rev[i])swap(a[i],a[rev[i]]);
      for(int k=1;k<n;k*=2)for(int i=0;i<n;i+=2*k)for(int j=0;j<k
          ;j++){
        cd z=rt[j+k]*a[i+j+k];
        a[i+j+k]=a[i+j]-z;
        a[i+j]+=z;
      }
    }
    template<class U>
    static db norm(const U &x){
      return INT?round(x):x;
    }
    static vt conv(const vt &a,const vt &b){
      if(a.empty()||b.empty())return {};
      vt res(a.size()+b.size()-1);
      int n=2;
      while(n<res.size())n<<=1;
      vc in(n),out(n);
      copy(a.begin(),a.end(),in.begin());
      for(int i=0;i<b.size();i++)in[i].imag(b[i]);
      fft(in);
      for(auto &x:in)x*=x;
      for(int i=0;i<n;i++)out[i]=in[-i&(n-1)]-conj(in[i]);
      fft(out);
      for(int i=0;i<res.size();i++)res[i]=norm(imag(out[i])/(4*n)
          );
      return res;
    }
    static vt convMod(const vt &a,const vt &b){
      assert(mod>0);
      if(a.empty()||b.empty())return {};
      vt res(a.size()+b.size()-1);
      int n=2;
      while(n<res.size())n<<=1;
      ll cut=int(sqrt(mod));
      vc in1(n),in2(n),out1(n),out2(n);
      for(int i=0;i<a.size();i++)in1[i]=cd(ll(a[i])/cut,ll(a[i])%
          cut); // a1 + i * a2
      for(int i=0;i<b.size();i++)in2[i]=cd(ll(b[i])/cut,ll(b[i])%
          cut); // b1 + i * b2
      fft(in1),fft(in2);
      for(int i=0;i<n;i++){
        int j=-i&(n-1);
        out1[j]=(in1[i]+conj(in1[j]))*in2[i]/(2.l*n); // f1 * (g1
            + i * g2) = f1 * g1 + i f1 * g2
        out2[j]=(in1[i]-conj(in1[j]))*in2[i]/cd(0.l,2.l*n); // f2
            * (g1 + i * g2) = f2 * g1 + i f2 * g2
      }
      fft(out1),fft(out2);
      for(int i=0;i<res.size();i++){
        ll x=round(real(out1[i])),y=round(imag(out1[i]))+round(
            real(out2[i])),z=round(imag(out2[i]));
```

```
      res[i]=((x%mod*cut+y)%mod*cut+z)%mod; // a1 * b1 * cut^2
          + (a1 * b2 + a2 * b1) * cut + a2 * b2
    }
    return res;
  }
  vt operator()(const vt &a,const vt &b){
    return mod>0?convMod(a,b):conv(a,b);
  }
};
```

ntt.hpp

Description: Number Theoretic Transform
Time: $\mathcal{O}(N \log N)$

"src/number-theory/binpow.hpp", "src/number-theory/montgomery-modint.hpp" 6b90cc, 43 lines

```
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root = 62). The last two are > 10^9.

template<class mint>
struct NTT{
  using vm = vector<mint>;

  static constexpr mint root=mint::get_root();
    static_assert(root!=0);

  static void ntt(vm &a){
    int n=a.size(),L=31-__builtin_clz(n);
    vm rt(n);
    rt[1]=1;
    for(int k=2,s=2;k<n;k*=2,s++){
      mint z[]={1,binpow(root,mint::get_mod()>>s)};
      for(int i=k;i<2*k;i++)rt[i]=rt[i/2]*z[i&1];
    }
    vector<int> rev(n);
    for(int i=1;i<n;i++)rev[i]=(rev[i/2]|(i&1)<<L)/2;
    for(int i=1;i<n;i++)if(i<rev[i])swap(a[i],a[rev[i]]);
    for(int k=1;k<n;k*=2)for(int i=0;i<n;i+=2*k)for(int j=0;j<k
        ;j++){
      mint z=rt[j+k]*a[i+j+k];
      a[i+j+k]=a[i+j]-z;
      a[i+j]+=z;
    }
  }
  static vm conv(const vm &a,const vm &b){
    if(a.empty()||b.empty())return {};
    int s=a.size()+b.size()-1,n=2;
    while(n<s)n<<=1;
    mint inv=mint(n).inv();
    vm in1(a),in2(b),out(n);
    in1.resize(n),in2.resize(n);
    ntt(in1),ntt(in2);
    for(int i=0;i<n;i++)out[-i&(n-1)]=in1[i]*in2[i]*inv;
    ntt(out);
    return vm(out.begin(),out.begin()+s);
  }
  vm operator()(const vm &a,const vm &b){
    return conv(a,b);
  }
};
```

lagrange-interpolate.hpp

Description: Given $f(0), \ldots, f(n-1)$ of a polynomial f of degree $< n$, return $f(c)$. Requires $n \leq$ mod.
Time: $\mathcal{O}(N)$

530ee1, 20 lines

```
template<class mint>
mint lagrange_interpolate(const vector<mint> &f,mint c){
    int n=f.size();
    if(n==0)return 0;
    if(c.val()<n)return f[c.val()];
    vector<mint> l(n+1),r(n+1),ifac(n);
    l[0]=r[n]=ifac[0]=1;
    for(int i=0;i<n;i++)l[i+1]=l[i]*(c-i);
    for(int i=n-1;i>=0;i--)r[i]=r[i+1]*(c-i);
    for(int i=1;i<n;i++)ifac[i]=ifac[i-1]*i;
    ifac[n-1]=ifac[n-1].inv();
    for(int i=n-1;i>1;i--)ifac[i-1]=ifac[i]*i;
    mint ans=0;
    for(int i=0;i<n;i++){
        mint cur=f[i]*ifac[i]*ifac[n-i-1];
        if((n-i-1)&1)cur*=-1;
        ans+=cur*l[i]*r[i+1];
    }
    return ans;
}
```

subproduct-tree.hpp

Description: Subproduct tree of points $x_0, \ldots, x_{m-1}$. Node i covers $[l, r)$ and stores $\prod_{l \leq j < r}(x - x_j)$, root is node 1.
Time: $\mathcal{O}\left(M \log^2 M\right)$

"src/polynomials/formal-power-series.hpp" 0e437e, 14 lines

```
template<class mint>
struct SubproductTree{
    int m;
    vector<mint> xs;
    vector<FormalPowerSeries<mint>> t;
    SubproductTree(const vector<mint> &xs)
        :m(xs.size()),xs(xs),t(4*m){if(m)build(1,0,m);}
    void build(int i,int l,int r){
        if(r-l==1)return void(t[i]={-xs[l],1});
        int mid=(l+r)/2;
        build(2*i,l,mid),build(2*i+1,mid,r);
        t[i]=t[2*i]*t[2*i+1];
    }
};
```

multipoint-evaluation.hpp

Description: Evaluate polynomial f at points $x_0, \ldots, x_{m-1}$.
Time: $\mathcal{O}\left(N \log^2 N\right)$ where $N = \max(|f|, M)$

"src/polynomials/subproduct-tree.hpp" 7df322, 22 lines

```
template<class mint>
vector<mint> multipoint_evaluation(
    const FormalPowerSeries<mint> &f,
    const SubproductTree<mint> &T){
    vector<mint> res(T.m);
    auto rec=[&](auto &&self,auto g,int i,int l,int r)->void {
        g%=T.t[i];
        if(r-l<=64){
            for(int j=l;j<r;j++)res[j]=g.eval(T.xs[j]);
            return;
        }
        int mid=(l+r)/2;
        self(self,g,2*i,l,mid),self(self,g,2*i+1,mid,r);
    };
    if(T.m)rec(rec,f,1,0,T.m);
    return res;
}
template<class mint>
vector<mint> multipoint_evaluation(
    const FormalPowerSeries<mint> &f,const vector<mint> &xs){
    return multipoint_evaluation(f,SubproductTree<mint>(xs));
}
```

polynomial-interpolation.hpp

Description: Polynomial f of degree $< m$ with $f(x_i) = y_i$, x_i must be distinct.
Time: $\mathcal{O}\left(M \log^2 M\right)$

"src/polynomials/multipoint-evaluation.hpp" 1f9e31, 15 lines

```
template<class mint>
FormalPowerSeries<mint> polynomial_interpolation(
    const vector<mint> &xs,const vector<mint> &ys){
    using FPS = FormalPowerSeries<mint>;
    if(xs.empty())return {};
    SubproductTree<mint> T(xs);
    auto w=multipoint_evaluation(T.t[1].diff(),T);
    auto rec=[&](auto &&self,int i,int l,int r)->FPS {
        if(r-l==1)return {ys[l]/w[l]};
        int mid=(l+r)/2;
        auto L=self(self,2*i,l,mid),R=self(self,2*i+1,mid,r);
        return L*T.t[2*i+1]+R*T.t[2*i];
    };
    return rec(rec,1,0,T.m);
}
```

taylor-shift.hpp

Description: Given $f(x)$, returns $f(x + c)$. Needs $N <$ mod.
Time: $\mathcal{O}(N \log N)$

"src/polynomials/formal-power-series.hpp" dde66f, 18 lines

```
template<class mint>
FormalPowerSeries<mint> taylor_shift(
    FormalPowerSeries<mint> f,mint c){
    int n=SZ(f);
    if(!n)return f;
    vector<mint> fac(n,1),ifac(n),b(n);
    for(int i=1;i<n;i++)fac[i]=fac[i-1]*mint(i);
    ifac[n-1]=fac[n-1].inv();
    for(int i=n-1;i>0;i--)ifac[i-1]=ifac[i]*mint(i);
    mint p=1;
    for(int i=0;i<n;i++){
        f[i]*=fac[i],b[i]=p*ifac[i],p*=c;
    }
    reverse(ALL(f));
    auto g=NTT<mint>::conv(f,b);
    for(int i=0;i<n;i++)f[i]=g[n-1-i]*ifac[i];
    return f;
}
```

berlekamp-massey.hpp

Description: Finds the shortest recurrence c of length L with $s_i = \sum_{j=1}^{L} c_{j-1} s_{i-j}$ for all $L \leq i < n$. Needs $2L$ terms to recover a recurrence of order L.
Usage: `berlekamp_massey(vector<mint>{0,1,1,3,5,11}) // {1,2}`
Time: $\mathcal{O}\left(N^2\right)$

75e0c8, 21 lines

```
template<class mint>
vector<mint> berlekamp_massey(const vector<mint> &s){
    int n=SZ(s),L=0,m=0;
    vector<mint> C(n+1),B(n+1),T;
    C[0]=B[0]=1;
    mint b=1;
    for(int i=0;i<n;i++){
        m++;
        mint d=s[i];
        for(int j=1;j<=L;j++)d+=C[j]*s[i-j];
        if(d==mint(0))continue;
        T=C;
        mint coef=d/b;
        for(int j=m;j<=n;j++)C[j]-=coef*B[j-m];
        if(2*L>i)continue;
        L=i+1-L,B=T,b=d,m=0;
    }
    vector<mint> c(L);
    for(int i=0;i<L;i++)c[i]=-C[i+1];
    return c;
```

```
}
```

linear-recurrence.hpp

Description: k-th term of $a_i = \sum_{j=1}^{L} c_{j-1} a_{i-j}$ given $a_0, \ldots, a_{L-1}$ (Bostan-Mori, $a_k = [x^k]P/Q$). Given only the first $2L$ terms s, combine with Berlekamp-Massey.
Usage: `linear_recurrence<mint>({0,1},{1,1},10) // 55`
`linear_recurrence(s,berlekamp_massey(s),k)`
Time: $\mathcal{O}(L \log L \log k)$

"src/polynomials/ntt.hpp" 255ddf, 28 lines

```
template<class mint>
mint linear_recurrence(const vector<mint> &a,
    const vector<mint> &c,ll k){
    if(k<SZ(a))return a[k];
    int L=SZ(c),n=2,h;
    if(!L)return 0;
    while(n<=2*L)n*=2;
    h=n/2;
    vector<mint> P(n),Q(n),A(n),B(n);
    Q[0]=1;
    for(int i=0;i<L;i++)Q[i+1]=-c[i];
    auto t=NTT<mint>::conv(vector<mint>(a.begin(),
        a.begin()+L),vector<mint>(Q.begin(),Q.begin()+L+1));
    copy(t.begin(),t.begin()+L,P.begin());
    mint iv=mint(n).inv();
    for(;k;k>>=1){
        NTT<mint>::ntt(P),NTT<mint>::ntt(Q);
        for(int i=0;i<n;i++){ // Q(-x) <-> index i^h
            A[-i&(n-1)]=P[i]*Q[i^h]*iv;
            B[-i&(n-1)]=Q[i]*Q[i^h]*iv;
        }
        NTT<mint>::ntt(A),NTT<mint>::ntt(B);
        fill(ALL(P),0),fill(ALL(Q),0);
        for(int i=0;i<L;i++)P[i]=A[2*i+(k&1)];
        for(int i=0;i<=L;i++)Q[i]=B[2*i];
    }
    return P[0];
}
```

Convolutions (7)

bitwise-conv.hpp

Description: Bitwise AND/OR/XOR convolution. $|A| = |B|$ must be equal and a power of 2. Superset zeta: $A'[S] = \sum_{T \supseteq S} A[T]$ (AND). Subset zeta: $A'[S] = \sum_{T \subseteq S} A[T]$ (OR). Mobius transforms are their inverses. XOR FWHT: $A'[S] = \sum_T (-1)^{|S \& T|} A[T]$.
Time: $\mathcal{O}(N \log N)$.

01a7e0, 39 lines

```
template<class T> void superset_zeta(vector<T> &a){
    int n=SZ(a);assert(n==(n&-n));
    for(int i=1;i<n;i<<=1)for(int j=n;j--;)if(j&i)a[j^i]+=a[j];
}
template<class T> void superset_mobius(vector<T> &a){
    int n=SZ(a);assert(n==(n&-n));
    for(int i=n;i>>=1;)for(int j=n;j--;)if(j&i)a[j^i]-=a[j];
}
template<class T> void subset_zeta(vector<T> &a){
    int n=SZ(a);assert(n==(n&-n));
    for(int i=1;i<n;i<<=1)for(int j=n;j--;)if(j&i)a[j]+=a[j^i];
}
template<class T> void subset_mobius(vector<T> &a){
    int n=SZ(a);assert(n==(n&-n));
    for(int i=n;i>>=1;)for(int j=n;j--;)if(j&i)a[j]-=a[j^i];
}
template<class T> void fwht(vector<T> &a){
    int n=SZ(a);assert(n==(n&-n));
    for(int i=1;i<n;i<<=1)for(int j=n;j--;)if(j&i){
        T &u=a[j^i],&v=a[j];tie(u,v)=make_pair(u+v,u-v);
    }
}
template<class T> vector<T> and_conv(vector<T> a,vector<T> b){
    superset_zeta(a);superset_zeta(b);
    for(int i=0;i<SZ(a);i++)a[i]*=b[i];
    superset_mobius(a);return a;
}
template<class T> vector<T> or_conv(vector<T> a,vector<T> b){
    subset_zeta(a);subset_zeta(b);
    for(int i=0;i<SZ(a);i++)a[i]*=b[i];
    subset_mobius(a);return a;
}
template<class T> vector<T> xor_conv(vector<T> a,vector<T> b){
    int n=SZ(a);fwht(a);fwht(b);
    for(int i=0;i<n;i++)a[i]*=b[i];
    fwht(a);T d=T(1)/T(n);
    if(d==T(0))for(auto &x:a)x/=n;else for(auto &x:a)x*=d;
    return a;
}
```

gcd-lcm-conv.hpp

Description: GCD/LCM convolution on indices $1..n-1$ (index 0 is ignored). Multiple zeta: $A'[n] = \sum_{n|m} A[m]$ (GCD). Divisor zeta: $A'[n] = \sum_{d|n} A[d]$ (LCM). Mobius transforms are their inverses.
Time: $\mathcal{O}(N \log \log N)$.

81232d, 30 lines

```
template<class T> void multiple_zeta(vector<T> &a){
    int n=SZ(a);vector<bool> pr(n,1);
    for(int p=2;p<n;p++)if(pr[p])
        for(int i=(n-1)/p;i>0;i--)pr[i*p]=0,a[i]+=a[i*p];
}
template<class T> void multiple_mobius(vector<T> &a){
    int n=SZ(a);vector<bool> pr(n,1);
    for(int p=2;p<n;p++)if(pr[p])
        for(int i=1;i*p<n;i++)pr[i*p]=0,a[i]-=a[i*p];
}
template<class T> void divisor_zeta(vector<T> &a){
    int n=SZ(a);vector<bool> pr(n,1);
    for(int p=2;p<n;p++)if(pr[p])
        for(int i=1;i*p<n;i++)pr[i*p]=0,a[i*p]+=a[i];
}
template<class T> void divisor_mobius(vector<T> &a){
    int n=SZ(a);vector<bool> pr(n,1);
    for(int p=2;p<n;p++)if(pr[p])
        for(int i=(n-1)/p;i>0;i--)pr[i*p]=0,a[i*p]-=a[i];
}
template<class T> vector<T> gcd_conv(vector<T> a,vector<T> b){
    multiple_zeta(a);multiple_zeta(b);
    for(int i=0;i<SZ(a);i++)a[i]*=b[i];
    multiple_mobius(a);return a;
}
template<class T> vector<T> lcm_conv(vector<T> a,vector<T> b){
    divisor_zeta(a);divisor_zeta(b);
    for(int i=0;i<SZ(a);i++)a[i]*=b[i];
    divisor_mobius(a);return a;
}
```

max-plus-convolution.hpp

Description: Max Plus Convolution. Find $C[k] = \max_{i+j=k}\{A[i] + B[j]\}$ for arbitrary A and concave B ($B[i] - B[i-1] \geq B[i+1] - B[i]$). smawck is the SMAWK algorithm for row-wise maxima of a totally monotone matrix; f(i,j,k) is true if $M[i][j] \leq M[i][k]$ (i.e. column k is at least as good as j, higher is better).
Time: $\mathcal{O}(N + M)$.

1048b4, 48 lines

```
template<class F>
vector<int> smawck(const F &f,const vector<int> &rows,
                   const vector<int> &cols){
    int n=SZ(rows),m=SZ(cols);
    if(max(n,m)<=2){
        vector<int> ans(n,-1);
        for(int i=0;i<n;i++)for(int j:cols)
            if(ans[i]==-1||f(rows[i],ans[i],j))ans[i]=j;
        return ans;
    }
    if(n<m){
        vector<int> st;
        for(int j:cols){
            while(SZ(st)&&f(rows[SZ(st)-1],st.back(),j))
                st.pop_back();
            if(SZ(st)<n)st.emplace_back(j);
        }
        return smawck(f,rows,st);
    }
    vector<int> ans(n,-1),nr;
    for(int i=1;i<n;i+=2)nr.emplace_back(rows[i]);
    auto res=smawck(f,nr,cols);
    for(int i=0;i<SZ(nr);i++)ans[2*i+1]=res[i];
    for(int i=0,l=0,r=0;i<n;i+=2){
        if(i+1==n)r=m;
        while(r<m&&cols[r]<=ans[i+1])r++;
        for(ans[i]=cols[l];l+1<r;)
            if(f(rows[i],ans[i],cols[++l]))ans[i]=cols[l];
    }
    return ans;
}
template<class F>
vector<int> smawck(const F &f,int n,int m){
    vector<int> r(n),c(m);iota(ALL(r),0);iota(ALL(c),0);
    return smawck(f,r,c);
}
template<class T>
vector<T> max_plus_convolution_arbitary_convex(vector<T> a,
        const vector<T> &b){
    int n=SZ(a),m=SZ(b);if(!n||!m)return {};
    if(m==1){for(auto &x:a)x+=b[0];return a;}
    auto f=[&](int i,int j){return a[j]+b[i-j];};
    auto cmp=[&](int i,int j,int k){
        return i>=k&&(i-j>=m||f(i,j)<=f(i,k));};
    auto best=smawck(cmp,n+m-1,n);vector<T> ans(n+m-1);
    for(int i=0;i<n+m-1;i++)ans[i]=f(i,best[i]);
    return ans;
}
```

Graph (8)

k-th-shortest-path.hpp

Description: K-th shortest path

386610, 65 lines

```
vector<ll> kth_shortest_path(vector<vector<pair<int,ll>>> &adj,
     int s,int t,int k){
    int n=adj.size();
    vector<vector<pair<int,ll>>> radj(n);
    for(int u=0;u<n;u++)for(auto [v,w]:adj[u])radj[v].
        emplace_back(u,w);
    using P = pair<ll,int>;
    priority_queue<P,vector<P>,greater<P>> pq;
    vector<ll> dist(n,LINF);
    vector<int> par(n,-1);
    pq.emplace(dist[t]=0,t);
    while(!pq.empty()){
        auto [d,u]=pq.top();pq.pop();
        if(d>dist[u])continue;
        for(auto [v,w]:radj[u])if(d+w<dist[v])pq.emplace(dist[v
            ]=d+w,v),par[v]=u;
    }
```

```
    if(dist[s]==LINF)return {};
    struct Node{
        ll val;
        int id,rank,l,r;
    };
    vector<Node> nodes;
    auto new_node=[&](ll val,int id,int rank,int l,int r){
        nodes.emplace_back(Node{val,id,rank,l,r});
        return (int)nodes.size()-1;
    };
    function<int(int,ll,int)> insert=[&](int t,ll val,int id){
        if(t==-1||val<nodes[t].val)return new_node(val,id,0,t
            ,-1);
        int l=nodes[t].l,r=insert(nodes[t].r,val,id);
        if(l==-1||nodes[r].rank>nodes[l].rank)swap(l,r);
        return new_node(nodes[t].val,nodes[t].id,r!=-1?nodes[r
            ].rank+1:0,l,r);
    };
    vector<int> ptr(n,-1);
    vector<vector<int>> ch(n);
    for(int i=0;i<n;i++)if(par[i]!=-1)ch[par[i]].emplace_back(i
        );
    queue<int> q;
    q.emplace(t);
    while(!q.empty()){
        int u=q.front();q.pop();
        bool f=false;
        for(auto [v,w]:adj[u])if(dist[v]<LINF){
            ll c=w+dist[v]-dist[u];
            if(!f&&!c&&v==par[u]){
                f=true;
                continue;
            }
            ptr[u]=insert(ptr[u],c,v);
        }
        for(auto v:ch[u]){
            ptr[v]=ptr[u];
            q.emplace(v);
        }
    }
    vector<ll> ans{dist[s]};
    if(ptr[s]==-1)return ans;
    pq.emplace(dist[s]+nodes[ptr[s]].val,ptr[s]);
    while(!pq.empty()&&ans.size()<k){
        auto [d,u]=pq.top();pq.pop();
        ans.emplace_back(d);
        ll val=nodes[u].val;
        int v=nodes[u].id,l=nodes[u].l,r=nodes[u].r;
        if(ptr[v]!=-1)pq.emplace(d+nodes[ptr[v]].val,ptr[v]);
        if(l!=-1)pq.emplace(d+nodes[l].val-val,l);
        if(r!=-1)pq.emplace(d+nodes[r].val-val,r);
    }
    return ans;
}
```

Tree (9)

link-cut-tree.hpp

Description: Link Cut Tree (1-indexed) 75e7e1, 58 lines

```
template<int N,class T>
struct LinkCutTree{
    int ch[N][2],par[N],rev[N];
    T val[N],sum[N],rsum[N],lz[N];
    void toggle(int v){
        if(!v)return;
        swap(ch[v][0],ch[v][1]),swap(sum[v],rsum[v]),rev[v]^=1;
    }
    void apply(int v,T x){
        if(!v)return;
        val[v]=val[v]+x,lz[v]=lz[v]+x;
        sum[v]=sum[v]+x,rsum[v]=rsum[v]+x;
    }
    void push(int v){
        if(rev[v])toggle(ch[v][0]),toggle(ch[v][1]),rev[v]=0;
        apply(ch[v][0],lz[v]),apply(ch[v][1],lz[v]),lz[v]=T();
    }
    void pull(int v){
        sum[v]=sum[ch[v][0]]+val[v]+sum[ch[v][1]];
        rsum[v]=rsum[ch[v][1]]+val[v]+rsum[ch[v][0]];
    }
    bool pos(int v){return ch[par[v]][1]==v;}
    bool is_root(int v){return ch[par[v]][pos(v)]!=v;}
    void rotate(int v){
        int u=par[v],g=par[u],x=pos(v);
        if(!is_root(u))ch[g][pos(u)]=v;
        if((ch[u][x]=ch[v][!x]))par[ch[u][x]]=u;
        ch[v][!x]=u,par[u]=v,par[v]=g,pull(u),pull(v);
    }
    void splay(int v){
        for(push(v);!is_root(v);rotate(v)){
            int u=par[v],r=is_root(u);
            if(!r)push(par[u]);
            push(u),push(v);
            if(!r)rotate(pos(u)==pos(v)?u:v);
        }
    }
    void access(int v){
        for(int u=v,c=0;u;c=u,u=par[u])
            splay(u),ch[u][1]=c,pull(u);
        splay(v);
    }
    void reroot(int v){access(v),toggle(v);}
    bool connected(int u,int v){
        reroot(u),access(v);return u==v||par[u];}
    void link(int u,int v){
        reroot(u),access(v);
        assert(u!=v&&!par[u]);par[u]=v;}
    void cut(int u,int v){
        reroot(u),access(v);
        assert(ch[v][0]==u&&!ch[u][0]&&!ch[u][1]);
        ch[v][0]=par[u]=0,pull(v);
    }
    T aggregate(int u,int v){
        reroot(u),access(v);return sum[v];}
    void add(int u,int v,T x){reroot(u),access(v),apply(v,x);}
    void set(int u,T v){reroot(u),val[u]=v,pull(u);}
};
```

euler-tour-tree.hpp

Description: Euler tour tree: a dynamic forest on vertices $1..n$ with subtree add / subtree sum. Each tree is its Euler tour (vertex nodes + a weight-0 node per directed edge) in a Treap. The subtree of v whose parent is p (adjacent) lies between the edge nodes (p,v) and (v,p). N > 3n (cut edge nodes are recycled).
Usage: `EulerTourTree<N,ll> ett; ett.init(n); ett.set(v,x); ett.link(u,v); ett.cut(u,v); ett.add(v,p,x); ett.query(v,p);`
Time: $\mathcal{O}(\log n)$ expected per operation

"src/data-structures/treap.hpp" fb0251, 55 lines

```
template<int N,class T>
struct EulerTourTree:Treap<N,T>{
    using B = Treap<N,T>;using B::t,B::make,B::split;
    using B::merge,B::root,B::pos,B::apply;
    map<pair<int,int>,int> E; // directed edge -> node
    vector<int> fr; // freed edge nodes
    void init(int n){for(int i=1;i<=n;i++)make(T());}
    int edge(){ // recycled or new weight-0 node
        if(fr.empty())return make(T(),0);
        int x=fr.back();fr.pop_back();
        return t[x]={0,0,0,1},x;
    }
    void reroot(int v){
        auto [a,b]=split(root(v),pos(v));
        merge(b,a);
    }
    void link(int u,int v){
        reroot(u),reroot(v);
        assert(root(u)!=root(v));
        int e=E[{u,v}]=edge(),f=E[{v,u}]=edge();
        merge(merge(merge(root(u),e),root(v)),f);
    }
    void cut(int u,int v){ // sides: y, and x+z minus edges
        auto [x,y,z,in]=range(v,u);
        fr.pb(E[{u,v}]),fr.pb(E[{v,u}]);
        E.erase({u,v}),E.erase({v,u});
        merge(split(x,t[x].sz-1).first,split(z,1).second);
    }
    // tour = X|Y|Z cut at edge nodes (p,v),(v,p); subtree of v
    // (parent p) is Y if (p,v) is first, else X and Z (cyclic)
    array<int,4> range(int v,int p){
        int a=pos(E[{p,v}]),b=pos(E[{v,p}]);
        auto [x,z]=split(root(v),max(a,b));
        auto [x1,y]=split(x,min(a,b)+1);
        return {x1,y,z,a<b};
    }
    void add(int v,int p,T w){
        auto [x,y,z,in]=range(v,p);
        if(in)apply(y,w);
        else apply(x,w),apply(z,w);
        merge(merge(x,y),z);
    }
    T query(int v,int p){
        auto [x,y,z,in]=range(v,p);
        T s=in?t[y].sum:t[x].sum+t[z].sum;
        merge(merge(x,y),z);
        return s;
    }
    void set(int v,T x){ // point set
        auto [a,b]=split(root(v),pos(v));
        auto [c,d]=split(b,1); // c == v
        t[c].val=t[c].sum=x;
        merge(a,merge(c,d));
    }
};
```

static-top-tree.hpp

Description: Static top tree of a tree with vertices $1..n$ (adj has size $n+1$, root 1). Node pool is 1-indexed, 0 = null, at most $2n-1$ nodes; vertex v is its own Vertex/AddVertex node. Balanced by subtree size, height $O(\log n)$. StaticTopTreeRerootingDP takes a DP struct D (interface in the code). After changing the data of v, call update(v).
Time: build $\mathcal{O}(n \log n)$, update / query_subtree / query_reroot $\mathcal{O}(\log n)$, query_all $\mathcal{O}(1)$

"src/contest/template.hpp" 82b99b, 117 lines

```
struct StaticTopTree{
    using P = pair<int,int>;
    using G = vector<vector<int>>;
    enum Type{Compress,Rake,AddVertex,Vertex};
    int n,cnt,root;
    G &adj;
    vector<int> hv,fa,l,r,par,t;
    StaticTopTree(G &g):adj(g){ // 1-indexed
        n=cnt=SZ(g)-1;
        hv=fa=l=r=par=t=vector<int>(2*n+1);
        vector<int> o{1},s(n+1,1);s[0]=0; // BFS order, sizes
        for(int i=0;i<SZ(o);i++){
```

```
            int u=o[i];
            for(int v:adj[u])if(v!=fa[u])fa[v]=u,o.pb(v);
        }
        for(int i=n-1;i>0;i--){
            int u=o[i],p=fa[u];s[p]+=s[u];
            if(s[u]>s[hv[p]])hv[p]=u;
        }
        root=compress(1).first;
    }
    int add(int i,int a,int b,Type ty){
        if(!i)i=++cnt;
        l[i]=a,r[i]=b,t[i]=ty,par[a]=par[b]=i;
        return i;
    }
    P merge(const vector<P> &a,int x,int y,Type ty){
        if(y-x==1)return a[x];
        int tot=0,m=x;
        for(int k=x;k<y;k++)tot+=a[k].second;
        while(tot>a[m].second)tot-=2*a[m++].second;
        auto [i,si]=merge(a,x,m,ty);
        auto [j,sj]=merge(a,m,y,ty);
        return {add(0,i,j,ty),si+sj};
    }
    P compress(int i){
        vector<P> a{add_vertex(i)};
        while(hv[i])a.eb(add_vertex(i=hv[i]));
        return merge(a,0,SZ(a),Compress);
    }
    P rake(int i){
        vector<P> a;
        for(int j:adj[i])
            if(j!=fa[i]&&j!=hv[i])a.eb(compress(j));
        return a.empty()?P(0,0):merge(a,0,SZ(a),Rake);
    }
    P add_vertex(int i){
        auto [j,s]=rake(i);
        return {add(i,j,0,j?AddVertex:Vertex),s+1};
    }
};
/* struct D{
    struct Path{static Path unit();};
    struct Point{static Point unit();};
    static Path vertex(int v),add_vertex(Point p,int v),
        compress(Path up,Path down);
    static Point add_edge(Path p),rake(Point a,Point b);
}; */
template<class D>
struct StaticTopTreeRerootingDP:StaticTopTree{
    using Path = typename D::Path;
    using Point = typename D::Point;
    vector<Path> path,rvpath;
    vector<Point> point;
    StaticTopTreeRerootingDP(G &g):StaticTopTree(g){
        path=rvpath=vector<Path>(cnt+1),point.resize(cnt+1);
        calc(root);
    }
    void pull(int u){
        int lc=l[u],rc=r[u];
        if(t[u]==Vertex)path[u]=rvpath[u]=D::vertex(u);
        else if(t[u]==Compress){
            path[u]=D::compress(path[lc],path[rc]);
            rvpath[u]=D::compress(rvpath[rc],rvpath[lc]);
        }
        else if(t[u]==Rake)point[u]=D::rake(pt(lc),pt(rc));
        else path[u]=rvpath[u]=D::add_vertex(pt(lc),u);
    }
    Point pt(int c){
        return t[c]==Rake?point[c]:D::add_edge(path[c]);}
    void calc(int u){if(u)calc(l[u]),calc(r[u]),pull(u);}
    void update(int u){for(;u;u=par[u])pull(u);}
    Path query_all(){return path[root];}
    Path query_subtree(int u){
        Path res=path[u];
        for(int p;(p=par[u])&&t[p]==Compress;u=p)
            if(l[p]==u)res=D::compress(res,path[r[p]]);
        return res;
    }
    pair<Path,Path> around(int u){ // {below u, rest of tree}
        Path dn=Path::unit(),up=dn;
        int p=par[u];
        for(;p&&t[p]==Compress;p=par[u=p])
            if(l[p]==u)dn=D::compress(dn,path[r[p]]);
            else up=D::compress(up,rvpath[l[p]]);
        if(p){
            Point s=Point::unit();
            for(;t[p]==Rake;p=par[u=p])
                s=D::rake(s,pt(l[p]^r[p]^u));
            s=D::rake(s,out(p));
            up=D::compress(up,D::add_vertex(s,p));
        }
        return {dn,up};
    }
    Point out(int u){
        auto [dn,up]=around(u);
        return D::rake(D::add_edge(dn),D::add_edge(up));
    }
    Path query_reroot(int u){
        Point s=out(u);
        if(t[u]==AddVertex)s=D::rake(s,pt(l[u]));
        return D::add_vertex(s,u);
    }
    Path query_side(int v,int w){// remove (v,w) then query(v)
        return fa[v]==w?query_subtree(v):around(w).second;
    }
};
```

dominator-tree.hpp

Description: Dominator tree of a directed graph from root r (Lengauer-Tarjan). Returns idom, with idom[r]=r and -1 for nodes unreachable from r. Helpers for common queries in namespace domtree.

Time: $\mathcal{O}\left((N+M)\log N\right)$

b52b23, 97 lines

```
// u dominates v: every path r->v passes through u.
// idom(v): the strict dominator of v closest to v.
// e(u) = DFS preorder index; the code works in this numbering.
// g: 0-indexed, g[u] = out-neighbours of u
vector<int> dominator_tree(const vector<vector<int>> &g,int r){
    int n=g.size(),t=0;
    vector<int> id(n,-1),ord,p(n),it(n),st{r};
    id[r]=t++,ord.push_back(r);
    while(!st.empty()){ // iterative DFS, p = DFS parent
        int u=st.back();
        if(it[u]==SZ(g[u]))st.pop_back();
        else if(int v=g[u][it[u]++];id[v]<0)
            p[id[v]=t++]=id[u],ord.pb(v),st.pb(v);
    }
    vector<vector<int>> rg(t),bk(t);
    for(int u=0;u<n;u++)if(id[u]>=0)
        for(int v:g[u])if(id[v]>=0)rg[id[v]].push_back(id[u]);
    vector<int> sd(t),lb(t),f(t),dom(t),path;
    iota(ALL(sd),0),lb=f=sd;
    auto eval=[&](int v){ // min sdom label on DSU path
        if(f[v]==v)return v;
        for(int x=v;f[f[x]]!=f[x];x=f[x])path.push_back(x);
        for(;!path.empty();path.pop_back()){
            int y=path.back();
            if(sd[lb[f[y]]]<sd[lb[y]])lb[y]=lb[f[y]];
            f[y]=f[f[y]];
        }
        return lb[v];
    };
    // sdom(u) = v with min e(v) having a path v=w0..wk=u with
    // e(wi)>e(u) for 0<i<k; it is a proper DFS-tree ancestor.
    // x(u) = node on tree path (sdom(u),u] with min sdom(x).
    // x(u)==u -> idom(u)=sdom(u), else idom(u)=idom(x(u)).
    for(int w=t-1;w>0;w--){
        for(int v:rg[w])sd[w]=min(sd[w],sd[eval(v)]);
        bk[sd[w]].push_back(w),f[w]=p[w];
        for(int v:bk[p[w]]){ // eval(v) = x(v)
            int u=eval(v);
            dom[v]=sd[u]<sd[v]?u:p[w];
        }
        bk[p[w]].clear();
    }
    vector<int> res(n,-1);
    res[r]=r;
    for(int w=1;w<t;w++){ // dom[w]=x(w) -> idom(x(w))
        if(dom[w]!=sd[w])dom[w]=dom[dom[w]];
        res[ord[w]]=ord[dom[w]];
    }
    return res;
}

// usage helpers, idom = dominator_tree(g,r)
namespace domtree{
    // children lists of the dominator tree (edge idom[v]->v)
    vector<vector<int>> children(const vector<int> &idom){
        int n=idom.size();
        vector<vector<int>> ch(n);
        for(int v=0;v<n;v++)
            if(idom[v]>=0&&idom[v]!=v)ch[idom[v]].pb(v);
        return ch;
    }
    // O(1) "u dominates v" via tin/tout on the dominator tree
    // (u is an ancestor of v), false if v is unreachable
    struct Dominates{
        vector<int> tin,tout;
        Dominates(const vector<int> &idom,int r){
            auto ch=children(idom);
            int n=idom.size(),T=0;
            tin.assign(n,-1),tout.assign(n,-1);
            vector<int> st{r},it(n);
            tin[r]=T++;
            while(!st.empty()){ // iterative DFS
                int u=st.back();
                if(it[u]==SZ(ch[u]))tout[u]=T++,st.pop_back();
                else{
                    int v=ch[u][it[u]++];
                    tin[v]=T++,st.pb(v);
                }
            }
        }
        bool operator()(int u,int v){
            if(tin[v]<0)return false;
            return tin[u]<=tin[v]&&tout[v]<=tout[u];
        }
    };
    // nodes on every r->t path (critical nodes), t up to r;
    // size = number of dominators of t, empty if unreachable
    vector<int> critical(const vector<int> &idom,int t){
        vector<int> res;
        if(idom[t]<0)return res;
        for(;;t=idom[t]){
            res.pb(t);
            if(idom[t]==t)break;
        }
        return res;
```

```
    }
}
```

Flows (10)

10.1 Flow and Matching Theorems

10.1.1 Cuts and covers

Max flow = min cut. Min cut: S = nodes reachable from s in the residual graph.

Bipartite $G = (L \cup R, E)$, ν = max matching, no isolated vertices for edge cover:

- **König:** min vertex cover = ν. Cover = (L not reachable) $\cup$ (R reachable) from s in the residual graph.
- Max independent set = $|V| - \nu$ (complement of a min vertex cover).
- Min edge cover = $|V| - \nu$ (matching + one edge per unmatched vertex).
- **Hall:** L can be fully matched iff $|N(S)| \geq |S|$ for all $S \subseteq L$. In general $\nu = |L| - \max_{S \subseteq L}(|S| - |N(S)|)$.

10.1.2 Path covers and Dilworth

DAG, split v into $v_{out} \in L$, $v_{in} \in R$, edge $u \to v$ becomes $u_{out} - v_{in}$: min vertex-disjoint path cover = $n - \nu$. Paths may share vertices: do the same on the transitive closure.

Dilworth: in a poset, min chain cover = max antichain = $n - \nu$ on the comparability DAG (closure). Max antichain: v with v_{out} reachable and v_{in} not reachable from s in the residual graph.

Mirsky: min antichain cover = longest chain.

10.1.3 Lower bounds and circulations

Edge $u \to v$ with $l \leq f \leq r$: add edge with capacity $r - l$, set $ex[v] \mathrel{+}= l$, $ex[u] \mathrel{-}= l$. New source S', sink T': $S' \to v$ (cap $ex[v]$) if $ex[v] > 0$, $v \to T'$ (cap $-ex[v]$) if $ex[v] < 0$.

- **Feasible circulation** iff maxflow(S', T') saturates all S' edges. Real flow = l+ flow.
- **s-t flow:** add $t \to s$ with cap ∞, check feasibility as above. Let f_0 = flow on $t \to s$; remove that edge. Max flow = f_0+ maxflow(s, t), min flow = f_0- maxflow(t, s) (on the residual graph).
- **Min cost circulation** (negative costs, no Bellman-Ford): saturate every negative edge ($f = cap$, update ex, its reverse edge gets cap cap, cost $-c$), then run min cost max flow $S' \to T'$. Add the saturated costs to the answer.

10.1.4 Max weight closure

Choose X closed under $u \in X \Rightarrow v \in X$ for each dependency $u \to v$, maximize $\sum_{x \in X} w_x$: $s \to v$ cap w_v ($w_v > 0$), $v \to t$ cap $-w_v$ ($w_v < 0$), $u \to v$ cap ∞. Answer = $\sum_{w_v > 0} w_v-$ mincut, X = source side.

dinic.hpp

Description: Dinic's Algorithm for finding the maximum flow. cut() returns the flow and res[i]=1 iff i is on the sink side.

Time: $\mathcal{O}(V^2 E)$ in general, $\mathcal{O}(VE \log U)$ with scaling where U is the maximum capacity.

6d200c, 61 lines

```
template<class T,bool directed=true,bool scaling=true>
struct Dinic{
    static constexpr T INF=numeric_limits<T>::max()/2;
    struct Edge{
        int to;T flow,cap;
        T remain(){return cap-flow;}
    };
    int n,s,t;T U;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> ptr,lv;
    bool calculated;T max_flow;
    Dinic(){}
    Dinic(int n,int s,int t){init(n,s,t);}
    void init(int _n,int _s,int _t){
        n=_n,s=_s,t=_t,U=0,calculated=false;
        e.clear(),g.assign(n,{});
    }
    void add_edge(int from,int to,T cap){
        assert(0<=from&&from<n&&0<=to&&to<n);
        g[from].pb(SZ(e)),e.pb({to,0,cap});
        g[to].pb(SZ(e)),e.pb({from,0,directed?0:cap});
        U=max(U,cap);
    }
    bool bfs(T scale){
        lv.assign(n,-1),lv[s]=0;
        vector<int> q{s};
        for(int i=0;i<SZ(q);i++)for(int j:g[q[i]]){
            int v=e[j].to;
            if(lv[v]==-1&&e[j].remain()>=scale)
                lv[v]=lv[q[i]]+1,q.pb(v);
        }
        return lv[t]!=-1;
    }
    T dfs(int u,int t,T f){
        if(u==t||f==0)return f;
        for(int &i=ptr[u];i<SZ(g[u]);i++){
            int j=g[u][i],v=e[j].to;
            if(lv[v]!=lv[u]+1)continue;
            T r=dfs(v,t,min(f,e[j].remain()));
            if(r>0)return e[j].flow+=r,e[j^1].flow-=r,r;
        }
        return 0;
    }
    T flow(){
        if(calculated)return max_flow;
        calculated=true,max_flow=0;
        T scale=scaling&&U?1LL<<(63-__builtin_clzll(U)):1LL;
        for(;scale>0;scale>>=1)while(bfs(scale)){
            ptr.assign(n,0);
            while(T f=dfs(s,t,INF))max_flow+=f;
        }
        return max_flow;
    }
    pair<T,vector<int>> cut(){
        flow();
        vector<int> res(n);
        for(int i=0;i<n;i++)res[i]=lv[i]==-1;
        return {max_flow,res};
    }
};
```

binary-optimization.hpp

Description: Binary Optimization. minimize $\kappa + \sum_i \theta_i(x_i) + \sum_{i<j} \phi_{ij}(x_i, x_j) + \sum_{i<j<k} \psi_{ijk}(x_i, x_j, x_k)$ where $x_i \in \{0, 1\}$ and ϕ_{ij}, ψ_{ijk} are submodular functions. a set function f is submodular if $f(S) + f(T) \geq f(S \cap T) + f(S \cup T)$ for all S, T. $\phi_{ij}(0,1) + \phi_{ij}(1,0) \geq \phi_{ij}(1,1) + \phi_{ij}(0,0)$.

"src/flows/dinic.hpp" 0d9c72, 95 lines

```
template<class T,bool minimize=true>
struct BinaryOptimization{
    static constexpr T INF=numeric_limits<T>::max()/2;
    int n,s,t,buf;
    T base;
    map<pair<int,int>,T> edges;
    BinaryOptimization(int _n):n(_n),s(n),t(n+1),buf(n+2),base
        (0){}
    void add_edge(int u,int v,T w){
        assert(w>=0);
        if(u==v||w==0)return;
        auto &e=edges[{u,v}];
        e=min(e+w,INF);
    }
    void add0(T w){
        base+=w;
    }
    void _add1(int i,T a,T b){
        if(a<=b){
            add0(a);
            add_edge(s,i,b-a);
        }else{
            add0(b);
            add_edge(i,t,a-b);
        }
    }
    void add1(int i,T x0,T x1){
        assert(0<=i&&i<n);
        if(!minimize)x0=-x0,x1=-x1;
        _add1(i,x0,x1);
    }
    void _add2(int i,int j,T a,T b,T c,T d){
        assert(b+c>=a+d);
        add0(a);
        _add1(i,0,c-a);
        _add1(j,0,d-c);
        add_edge(i,j,b+c-a-d);
    }
    void add2(int i,int j,T x00,T x01,T x10,T x11){
        assert(i!=j&&0<=i&&i<n&&0<=j&&j<n);
        if(!minimize)x00=-x00,x01=-x01,x10=-x10,x11=-x11;
        _add2(i,j,x00,x01,x10,x11);
    }
    void _add3(int i,int j,int k,T a,T b,T c,T d,T e,T f,T g,T
        h){
        T p=a+d+f+g-b-c-e-h;
        if(p>=0){
            add0(a);
            _add1(i,0,f-b);
            _add1(j,0,g-e);
            _add1(k,0,d-c);
            _add2(i,j,0,c+e-a-g,0,0);
            _add2(i,k,0,0,b+e-a-f,0);
            _add2(j,k,0,b+c-a-d,0,0);
            int u=buf++;
            add0(-p);
            add_edge(i,u,p);
            add_edge(j,u,p);
            add_edge(k,u,p);
            add_edge(u,t,p);
        }else{
            add0(h);
            _add1(i,c-g,0);
```

```
            _add1(j,b-d,0);
            _add1(k,e-f,0);
            _add2(i,j,0,0,d+f-b-h,0);
            _add2(i,k,0,d+g-c-h,0,0);
            _add2(j,k,0,0,f+g-e-h,0);
            int u=buf++;
            add0(p);
            add_edge(s,u,-p);
            add_edge(u,i,-p);
            add_edge(u,j,-p);
            add_edge(u,k,-p);
        }
    }
    void add3(int i,int j,int k,T x000,T x001,T x010,T x011,T
         x100,T x101,T x110,T x111){
        assert(i!=j&&j!=k&&k!=i&&0<=i&&i<n&&0<=j&&j<n&&0<=k&&k<
             n);
        if(!minimize){
            x000=-x000,x001=-x001,x010=-x010,x011=-x011;
            x100=-x100,x101=-x101,x110=-x110,x111=-x111;
        }
        _add3(i,j,k,x000,x001,x010,x011,x100,x101,x110,x111);
    }
    pair<T,vector<int>> solve(){
        Dinic<T> dinic(buf,s,t);
        for(auto &[p,w]:edges){
            auto [u,v]=p;
            dinic.add_edge(u,v,w);
        }
        auto [ans,cut]=dinic.cut();
        ans+=base;
        ans=min(ans,INF);
        cut.resize(n);
        return {minimize?ans:-ans,cut};
    }
};
```

k-ary-optimization.hpp

Description: k-ary Optimization. minimize $\kappa + \sum_i \theta_i(x_i) + \sum_{i<j} \phi_{ij}(x_i, x_j)$ where $x_i \in \{0, 1, \ldots, k-1\}$ and $\phi_{i,j}$ is monge. A function f is monge if $f(a,c) + f(b,d) \leq f(a,d) + f(b,c)$ for all $a < b$ and $c < d$. $\phi_{ij}(x-1,y) + \phi_{ij}(x,y+1) \leq \phi_{ij}(x-1,y+1) + \phi_{ij}(x,y)$. $\phi_{ij}(x,y) + \phi_{ij}(x-1,y+1) - \phi_{ij}(x-1,y) - \phi_{ij}(x,y+1) \geq 0$.

"src/flows/dinic.hpp" 5139ce, 88 lines

```
template<class T,bool minimize=true>
struct K_aryOptimization{
    static constexpr T INF=numeric_limits<T>::max()/2;
    int n,s,t,buf;
    T base;
    vector<int> ks;
    vector<vector<int>> id;
    map<pair<int,int>,T> edges;
    K_aryOptimization(int n,int k){init(vector<int>(n,k));}
    K_aryOptimization(const vector<int> &_ks){init(_ks);}
    void init(const vector<int> &_ks){
        ks=_ks;
        n=ks.size();
        s=0,t=1,buf=2;
        base=0;
        id.clear();
        edges.clear();
        for(auto &k:ks){
            assert(k>=1);
            vector<int> a(k+1);
            a[0]=s,a[k]=t;
            for(int i=1;i<k;i++)a[i]=buf++;
            id.emplace_back(a);
            for(int i=2;i<k;i++)add_edge(a[i],a[i-1],INF);
        }
    }
    void add_edge(int u,int v,T w){
        assert(w>=0);
        if(u==v||w==0)return;
        auto &e=edges[{u,v}];
        e=min(e+w,INF);
    }
    void add0(T w){
        base+=w;
    }
    void _add1(int i,vector<T> cost){
        add0(cost[0]);
        for(int j=1;j<ks[i];j++){
            T x=cost[j]-cost[j-1];
            if(x>0)add_edge(id[i][j],t,x);
            if(x<0)add0(x),add_edge(s,id[i][j],-x);
        }
    }
    void add1(int i,vector<T> cost){
        assert(0<=i&&i<n&&(int)cost.size()==ks[i]);
        if(!minimize)for(auto &x:cost)x=-x;
        _add1(i,cost);
    }
    void _add2(int i,int j,vector<vector<T>> cost){
        int h=ks[i],w=ks[j];
        _add1(j,cost[0]);
        for(int x=h-1;x>=0;x--)for(int y=0;y<w;y++)cost[x][y]-=
            cost[0][y];
        vector<T> a(h);
        for(int x=0;x<h;x++)a[x]=cost[x][w-1];
        _add1(i,a);
        for(int x=0;x<h;x++)for(int y=0;y<w;y++)cost[x][y]-=a[x
            ];
        for(int x=1;x<h;x++){
            for(int y=0;y<w-1;y++){
                T w=cost[x][y]+cost[x-1][y+1]-cost[x-1][y]-cost
                    [x][y+1];
                assert(w>=0); // monge
                add_edge(id[i][x],id[j][y+1],w);
            }
        }
    }
    void add2(int i,int j,vector<vector<T>> cost){
        assert(0<=i&&i<n&&0<=j&&j<n&&i!=j);
        assert((int)cost.size()==ks[i]);
        for(auto &v:cost)assert((int)v.size()==ks[j]);
        if(!minimize)for(auto &v:cost)for(auto &x:v)x=-x;
        _add2(i,j,cost);
    }
    pair<T,vector<int>> solve(){
        Dinic<T> dinic(buf,s,t);
        for(auto &[p,w]:edges){
            auto [u,v]=p;
            dinic.add_edge(u,v,w);
        }
        auto [val,cut]=dinic.cut();
        val+=base;
        if(!minimize)val=-val;
        vector<int> ans(n);
        for(int i=0;i<n;i++){
            ans[i]=ks[i]-1;
            for(int j=1;j<ks[i];j++)ans[i]-=cut[id[i][j]];
        }
        return {val,ans};
    }
};
```

min-cost-flow.hpp

Description: Minimum-cost maximum flow (successive shortest paths, Dijkstra with potentials; Bellman-Ford initializes the potentials if there are negative costs). No negative cycles.

Time: $\mathcal{O}(FE \log V)$ where F is max flow.

2838f9, 56 lines

```
template<class F,class C>
struct MinCostFlow{
    struct Edge{
        int to;F flow,cap;C cost;
        F getcap(){return cap-flow;}
    };
    int n;
    vector<Edge> e;
    vector<vector<int>> adj;
    vector<C> pot,dist;
    vector<int> pre;
    bool neg;
    const F FINF=numeric_limits<F>::max()/2;
    const C CINF=numeric_limits<C>::max()/2;
    MinCostFlow(){}
    MinCostFlow(int _n){init(_n);}
    void init(int _n){n=_n,neg=0,e.clear(),adj.assign(n,{});}
    void addEdge(int u,int v,F cap,C cost){
        adj[u].pb(SZ(e)),e.pb({v,0,cap,cost});
        adj[v].pb(SZ(e)),e.pb({u,0,0,-cost});
        if(cost<0)neg=true;
    }
    bool dijkstra(int s,int t){
        using P = pair<C,int>;
        dist.assign(n,CINF),pre.assign(n,-1);
        PQ<P> pq;
        dist[s]=0,pq.emplace(0,s);
        while(!pq.empty()){
            auto [d,u]=pq.top();pq.pop();
            if(dist[u]<d)continue;
            for(int i:adj[u]){
                C nd=d+pot[u]-pot[e[i].to]+e[i].cost;
                if(e[i].getcap()>0&&chmin(dist[e[i].to],nd))
                    pre[e[i].to]=i,pq.emplace(nd,e[i].to);
            }
        }
        return dist[t]<CINF;
    }
    pair<F,C> flow(int s,int t){
        F flow=0;C cost=0;
        pot.assign(n,0);
        if(neg)for(int k=0;k<n;k++)for(int i=0;i<SZ(e);i++)
            if(int u=e[i^1].to,v=e[i].to;e[i].getcap()>0)
                pot[v]=min(pot[v],pot[u]+e[i].cost);
        while(dijkstra(s,t)){
            for(int i=0;i<n;i++)pot[i]+=dist[i]<CINF?dist[i]:0;
            F aug=FINF;
            for(int u=t;u!=s;u=e[pre[u]^1].to)
                aug=min(aug,e[pre[u]].getcap());
            for(int u=t;u!=s;u=e[pre[u]^1].to)
                e[pre[u]].flow+=aug,e[pre[u]^1].flow-=aug;
            flow+=aug,cost+=aug*(pot[t]-pot[s]);
        }
        return {flow,cost};
    }
};
```

hopcroft-karp.hpp

Description: Fast bipartite matching algorithm. Left $[0,n)$, right $[0,m)$; after max_matching(), match[i] ($i < n$) is n+(matched right vertex) or -1.

Time: $\mathcal{O}\left(E\sqrt{V}\right)$

de4ced, 35 lines

```
struct HopcroftKarp{
```

```
    int n,m;
    vector<int> match,lv,ptr;
    vector<vector<int>> adj;
    HopcroftKarp(){}
    HopcroftKarp(int _n,int _m){init(_n,_m);}
    void init(int _n,int _m){n=_n,m=_m,adj.assign(n+m,{});}
    void add_edge(int u,int v){adj[u].pb(v+n);}
    void bfs(){
        lv.assign(n,-1);
        vector<int> q;
        for(int i=0;i<n;i++)if(match[i]==-1)lv[i]=0,q.pb(i);
        for(int i=0;i<SZ(q);i++)for(int v:adj[q[i]])
            if(int w=match[v];w!=-1&&lv[w]==-1)
                lv[w]=lv[q[i]]+1,q.pb(w);
    }
    bool dfs(int u){
        for(int &i=ptr[u];i<SZ(adj[u]);i++){
            int v=adj[u][i],w=match[v];
            if(w==-1||(lv[w]==lv[u]+1&&dfs(w)))
                return match[u]=v,match[v]=u,true;
        }
        return false;
    }
    int max_matching(){
        int ans=0,c=1;
        match.assign(n+m,-1);
        while(c){
            ptr.assign(n,0),bfs(),c=0;
            for(int i=0;i<n;i++)c+=match[i]==-1&&dfs(i);
            ans+=c;
        }
        return ans;
    }
};
```

hungarian-algorithm.hpp

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j] and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$.
Time: $\mathcal{O}\left(N^2M\right)$

2540b8, 34 lines

```
pair<ll, vector<int>> hungarian(const vector<vector<ll>> &a) {
  if (a.empty()) return {0, {}};
  int n = a.size() + 1, m = a[0].size() + 1;
  vector<ll> u(n), v(m);
    vector<int> p(m), ans(n - 1);
  for(int i=1;i<n;i++) {
    p[0] = i;
    int j0 = 0; // add "dummy" worker 0
    vector<ll> dist(m, LLONG_MAX);
        vector<int> pre(m, -1);
    vector<bool> done(m + 1);
    do { // dijkstra
      done[j0] = true;
      int i0 = p[j0], j1;
            ll delta = LLONG_MAX;
      for(int j=1;j<m;j++) if (!done[j]) {
        auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
        if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
        if (dist[j] < delta) delta = dist[j], j1 = j;
      }
      for(int j=0;j<m;j++) {
        if (done[j]) u[p[j]] += delta, v[j] -= delta;
        else dist[j] -= delta;
      }
      j0 = j1;
    } while (p[j0]);
    while (j0) { // update alternating path
      int j1 = pre[j0];
      p[j0] = p[j1], j0 = j1;
    }
  }
  for(int j=1;j<m;j++) if (p[j]) ans[p[j] - 1] = j - 1;
  return {-v[0], ans}; // min cost
}
```

general-matching.hpp

Description: General matching using blossom algorithm (1-indexed).
Time: $\mathcal{O}\left(EV\alpha(V)\right)$.

6c9e1a, 92 lines

```
template<int N>
struct GeneralMatching{

int n,m;
vector<int> adj[N+1];
int fa[N+1],link[N+1],match[N+1],vis[N+1],dep[N+1];
queue<int> q;

int find(int u){
    while(fa[u]!=u)u=fa[u]=fa[fa[u]];
    return u;
}
int get_lca(int u,int v){
    u=find(u),v=find(v);
    while(u!=v){
        if(dep[u]<dep[v])swap(u,v);
        u=find(link[match[u]]);
    }
    return u;
}
void blossom(int u,int v,int lca){
    while(find(u)!=lca){
        link[u]=v;
        v=match[u];
        if(vis[v]==0){
            vis[v]=1;
            q.emplace(v);
        }
        fa[u]=fa[v]=lca;
        u=link[v];
    }
}
bool augment(int st){
    for(int i=1;i<=n;i++)fa[i]=i;
    for(int i=1;i<=n;i++)vis[i]=-1;
    vis[st]=1;
    dep[st]=0;
    q=queue<int>();
    q.emplace(st);
    while(!q.empty()){
        int u=q.front();
        q.pop();
        for(auto v:adj[u]){
            if(vis[v]==-1){
                vis[v]=0;
                link[v]=u;
                dep[v]=dep[u]+1;
                if(!match[v]){
                    for(int x=v,y=u,z;y;y=link[x=z]){
                        z=match[y];
                        match[x]=y;
                        match[y]=x;
                    }
                    return 1;
                }
                vis[match[v]]=1;
                dep[match[v]]=dep[u]+2;
                q.emplace(match[v]);
            }else if(vis[v]==1&&find(u)!=find(v)){
                int lca=get_lca(u,v);
                blossom(u,v,lca);
                blossom(v,u,lca);
            }
        }
    }
    return 0;
}
int solve(){
    int ans=0;
    for(int u=1;u<=n;u++)if(!match[u]){
        for(auto v:adj[u])if(!match[v]){
            match[u]=v;
            match[v]=u;
            ans++;
            break;
        }
    }
    for(int u=1;u<=n;u++)if(!match[u])ans+=augment(u);
    return ans;
}
void init(int _n){
    n=_n;
    for(int i=1;i<=n;i++)match[i]=0;
    for(int i=1;i<=n;i++)adj[i].clear();
}
void add_edge(int u,int v){
    if(u==v)return;
    adj[u].emplace_back(v);
    adj[v].emplace_back(u);
}

};
```

general-weighted-matching.hpp

Description: General weighted matching using blossom algorithm (1-indexed).
Time: $\mathcal{O}\left(|V|^3\right)$ (fast in practice)

62fc30, 163 lines

```
template<int N>
struct GeneralWeightedMatching{
static const int INF=1e9+100;
struct Edge{
  int u,v,w;
  Edge(){}
  Edge(int u,int v,int w):u(u),v(v),w(w){}
};
int n,n_x;
Edge g[N*2][N*2];
int lab[N*2];
int match[N*2],slack[N*2],st[N*2],pa[N*2];
int flo_from[N*2][N+1],S[N*2],vis[N*2];
vector<int> flo[N*2];
queue<int> q;
int e_delta(const Edge &e){return lab[e.u]+lab[e.v]-g[e.u][e.v
    ].w*2;}
void update_slack(int u,int x){
  if(!slack[x]||e_delta(g[u][x])<e_delta(g[slack[x]][x]))slack[
      x]=u;
}
void set_slack(int x){
  slack[x]=0;
  for(int u=1;u<=n;u++)
    if(g[u][x].w>0&&st[u]!=x&&S[st[u]]==0)update_slack(u,x);
}
void q_push(int x){
```

```
  if(x<=n)q.push(x);
  else for(auto xs:flo[x])q_push(xs);
}
void set_st(int x,int b){
  st[x]=b;
  if(x>n)for(auto xs:flo[x])set_st(xs,b);
}
int get_pr(int b,int xr){
  int pr=find(ALL(flo[b]),xr)-flo[b].begin();
  if(pr&1){reverse(1+ALL(flo[b]));return SZ(flo[b])-pr;}
  return pr;
}
void set_match(int u,int v){
  Edge e=g[u][v];match[u]=g[u][v].v;if(u<=n)return;
  int xr=flo_from[u][e.u],pr=get_pr(u,xr);
  for(int i=0;i<pr;i++)set_match(flo[u][i],flo[u][i^1]);
  set_match(xr,v);rotate(flo[u].begin(),pr+ALL(flo[u]));
}
void augment(int u,int v){
  while(true){
    int xnv=st[match[u]];set_match(u,v);
    if(!xnv)return;
    set_match(xnv,st[pa[xnv]]);u=st[pa[xnv]];v=xnv;
  }
}
int get_lca(int u,int v){
  static int t=0;
  for(++t;u||v;swap(u,v)){
    if(u==0)continue;
    if(vis[u]==t)return u;
    vis[u]=t;u=st[match[u]];
    if(u)u=st[pa[u]];
  }
  return 0;
}
void add_blossom(int u,int lca,int v){
  int b=n+1;while(b<=n_x&&st[b])++b;
  if(b>n_x)++n_x; //new blossom;
  lab[b]=0;S[b]=0;match[b]=match[lca];flo[b]=vector<int>{lca};
  for(int x=u,y;x!=lca;x=st[pa[y]])
    flo[b].eb(x),flo[b].eb(y=st[match[x]]),q_push(y);
  reverse(1+ALL(flo[b]));
  for(int x=v,y;x!=lca;x=st[pa[y]])
    flo[b].eb(x),flo[b].eb(y=st[match[x]]),q_push(y);
  set_st(b,b);
  for(int x=1;x<=n_x;x++)g[b][x].w=g[x][b].w=0;
  for(int x=1;x<=n;x++)flo_from[b][x]=0;
  for(auto xs:flo[b]){
    for(int x=1;x<=n_x;x++)
      if(g[b][x].w==0||e_delta(g[xs][x])<e_delta(g[b][x]))
        g[b][x]=g[xs][x],g[x][b]=g[x][xs];
    for(int x=1;x<=n;x++)if(flo_from[xs][x])flo_from[b][x]=xs;
  }
  set_slack(b);
}
void expand_blossom(int b){
  for(auto xs:flo[b])set_st(xs,xs);
  int xr=flo_from[b][g[b][pa[b]].u],pr=get_pr(b,xr);
  for(int i=0;i<pr;i+=2){
    int xs=flo[b][i],xns=flo[b][i+1];
    pa[xs]=g[xns][xs].u;S[xs]=1;S[xns]=0;
    slack[xs]=0,set_slack(xns);q_push(xns);
  }
  S[xr]=1,pa[xr]=pa[b];
  for(int i=pr+1;i<SZ(flo[b]);i++)S[flo[b][i]]=-1,set_slack(flo
      [b][i]);
  st[b]=0;
}
bool on_found_edge(const Edge &e){
  int u=st[e.u],v=st[e.v];
  if(S[v]==-1){
    pa[v]=e.u;S[v]=1;
    int nu=st[match[v]];
    slack[v]=slack[nu]=S[nu]=0;q_push(nu);
  }else if(S[v]==0){
    int lca=get_lca(u,v);
    if(!lca)return augment(u,v),augment(v,u),true;
    add_blossom(u,lca,v);
  }
  return false;
}
bool matching(){
  for(int i=1;i<=n_x;i++)S[i]=-1;
  for(int i=1;i<=n_x;i++)slack[i]=-0;
  q=queue<int>();
  for(int x=1;x<=n_x;x++)if(st[x]==x&&!match[x])pa[x]=S[x]=0,
      q_push(x);
  if(q.empty())return false;
  while(true){
    while(!q.empty()){
      int u=q.front();q.pop();if(S[st[u]]==1)continue;
      for(int v=1;v<=n;v++)if(g[u][v].w>0&&st[u]!=st[v]){
        if(e_delta(g[u][v])==0){if(on_found_edge(g[u][v]))
            return true;}
        else update_slack(u,st[v]);
      }
    }
    int d=INF;
    for(int b=n+1;b<=n_x;b++)if(st[b]==b&&S[b]==1)d=min(d,lab[b
        ]/2);
    for(int x=1;x<=n_x;x++)if(st[x]==x&&slack[x]){
      if(S[x]==-1)d=min(d,e_delta(g[slack[x]][x]));
      else if(S[x]==0)d=min(d,e_delta(g[slack[x]][x])/2);
    }
    for(int u=1;u<=n;u++){
      if(S[st[u]]==0){if(lab[u]<=d)return 0;lab[u]-=d;}
      else if(S[st[u]]==1)lab[u]+=d;
    }
    for(int b=n+1;b<=n_x;b++)if(st[b]==b){
      if(S[st[b]]==0)lab[b]+=d*2;
      else if(S[st[b]]==1)lab[b]-=d*2;
    }
    q=queue<int>();
    for(int x=1;x<=n_x;x++)
      if(st[x]==x&&slack[x]&&st[slack[x]]!=x&&e_delta(g[slack[x
          ]][x])==0)
        if(on_found_edge(g[slack[x]][x]))return true;
    for(int b=n+1;b<=n_x;b++)
      if(st[b]==b&&S[b]==1&&lab[b]==0)expand_blossom(b);
  }
  return false;
}
pair<long long,int> solve(){
  for(int i=1;i<=n;i++)match[i]=0;
  n_x=n;int n_matches=0,w_max=0;long long tot_weight=0;
  for(int u=0;u<=n;u++)st[u]=u,flo[u].clear();
  for(int u=1;u<=n;u++)for(int v=1;v<=n;v++)
    flo_from[u][v]=u==v?u:0,w_max=max(w_max,g[u][v].w);
  for(int u=1;u<=n;u++)lab[u]=w_max;
  while(matching())++n_matches;
  for(int u=1;u<=n;u++)if(match[u]&&match[u]<u)tot_weight+=g[u
      ][match[u]].w;
  return make_pair(tot_weight,n_matches);
}
void add_edge(int u,int v,int w){
  if(w<=g[u][v].w)return;
  g[u][v].w=g[v][u].w=w;
}
void init(int _n){
  n=_n;
  for(int u=1;u<=n;u++)for(int v=1;v<=n;v++)g[u][v]=Edge(u,v,0)
      ;
}

};
```

String (11)

suffix-array.hpp

Description: Suffix Array. 58c0a5, 39 lines

```
template<class STR>
struct SuffixArray{
    int n;
    vector<int> sa,isa,lcp;
    // SparseTable<MinMonoid<int>> st;
    SuffixArray(){}
    SuffixArray(const STR &s){init(s);}
    void init(const STR &s){
        n=(int)s.size();
        sa=isa=lcp=vector<int>(n+1);
        sa[0]=n;
        iota(sa.begin()+1,sa.end(),0);
        sort(sa.begin()+1,sa.end(),[&](int i,int j){return s[i
            ]<s[j];});
        for(int i=1;i<=n;i++){
            int x=sa[i-1],y=sa[i];
            isa[y]=i>1&&s[x]==s[y]?isa[x]:i;
        }
        for(int len=1;len<=n;len<<=1){
            vector<int> ps(sa),pi(isa),pos(n+1);
            iota(pos.begin(),pos.end(),0);
            for(auto i:ps)if((i-=len)>=0)sa[pos[isa[i]]++]=i;
            for(int i=1;i<=n;i++){
                int x=sa[i-1],y=sa[i];
                isa[y]=pi[x]==pi[y]&&pi[x+len]==pi[y+len]?isa[x
                    ]:i;
            }
        }
        for(int i=0,k=0;i<n;i++){
            for(int j=sa[isa[i]-1];j+k<n&&s[j+k]==s[i+k];k++);
            lcp[isa[i]]=k;
            if(k)k--;
        }
        // st.init(lcp);
    }
    // int get_lcp(int i,int j){
    //     if(i==j)return n-i;
    //     auto [l,r]=minmax(isa[i],isa[j]);
    //     return st.query(l+1,r);
    // }
};
```

suffix-automaton.hpp

Description: Suffix Automaton on a node pool (root = 1, 0 = null). Call extend(c) with c in [0,A) for each character. Instance must be global; N > 2—s—. For a large alphabet use map<int,int> nxt[N].
Time: $\mathcal{O}(|s| \cdot A)$ 23595f, 74 lines

```
template<int N,int A=26>
struct SuffixAutomaton{
    int nxt[N][A],link[N],len[N],occ[N],cnt=1,last=1;
    void extend(int c){
        int cur=++cnt,p=last;
        len[cur]=len[last]+1,occ[cur]=1;
        for(;p&&!nxt[p][c];p=link[p])nxt[p][c]=cur;
```

```
        if(!p)link[cur]=1;
        else if(int q=nxt[p][c];len[p]+1==len[q])link[cur]=q;
        else{
            int r=++cnt;
            len[r]=len[p]+1,link[r]=link[q];
            copy(nxt[q],nxt[q]+A,nxt[r]);
            for(;p&&nxt[p][c]==q;p=link[p])nxt[p][c]=r;
            link[q]=link[cur]=r;
        }
        last=cur;
    }
    // ——— usages ———
    // # distinct non-empty substrings
    ll distinct_substrings(){
        ll res=0;
        for(int i=2;i<=cnt;i++)res+=len[i]-len[link[i]];
        return res;
    }
    // states sorted by len (shortest first), counting sort;
    // link[u] is shorter than u, nxt[u][c] is longer
    vector<int> order(){
        vector<int> c(len[last]+1),o(cnt);
        for(int i=1;i<=cnt;i++)c[len[i]]++;
        for(int i=1;i<SZ(c);i++)c[i]+=c[i-1];
        for(int i=cnt;i>=1;i--)o[--c[len[i]]]=i;
        return o;
    }
    // occ[u] = # occurrences (endpos size) of u's strings,
    // call once after building
    void calc_occ(){
        auto o=order();
        for(int i=cnt-1;i>0;i--)occ[link[o[i]]]+=occ[o[i]];
    }
    // # occurrences of non-empty t in s (needs calc_occ)
    int count(const string &t){
        int u=1;
        for(char x:t)if(!(u=nxt[u][x-'a']))return 0;
        return occ[u];
    }
    // longest common substring of s and t
    int lcs(const string &t){
        int u=1,l=0,res=0;
        for(char x:t){
            int c=x-'a';
            while(u>1&&!nxt[u][c])u=link[u],l=len[u];
            if(nxt[u][c])u=nxt[u][c],l++;
            res=max(res,l);
        }
        return res;
    }
    // k-th (1-indexed) lexicographically smallest distinct
    // substring, needs 1 <= k <= distinct_substrings()
    string kth(ll k){
        auto o=order();
        vector<ll> dp(cnt+1,1); // # paths from u (incl. empty)
        for(int i=cnt-1;i>=0;i--)for(int c=0;c<A;c++)
            if(int v=nxt[o[i]][c])dp[o[i]]+=dp[v];
        string res;
        for(int u=1;k>0;){
            for(int c=0;c<A;c++)if(int v=nxt[u][c]){
                if(dp[v]>=k){res+=char('a'+c),u=v,k--;break;}
                k-=dp[v];
            }
        }
        return res;
    }
};
```

aho-corasick.hpp

Description: Aho-Corasick on a node pool (root = 1, 0 = sentinel). insert() returns the end node of a pattern. After build(), ch is the full automaton and val[u] = sum of val over all patterns that are suffixes of u. Instance must be global; N > total pattern length.
Time: $\mathcal{O}(A \cdot N)$ build

6b809e, 30 lines

```
template<int N,class T,int A=26>
struct AhoCorasick{
    int ch[N][A],fail[N],cnt=1;
    T val[N];
    int insert(const string &s,T v){
        int u=1;
        for(char x:s){
            int &w=ch[u][x-'a'];
            if(!w)w=++cnt;
            u=w;
        }
        val[u]+=v;
        return u;
    }
    void build(){
        fill(ch[0],ch[0]+A,1); // sentinel: every edge to root
        vector<int> q{1};
        for(int i=0;i<SZ(q);i++){
            int u=q[i];
            for(int c=0;c<A;c++){
                int &v=ch[u][c];
                if(!v)v=ch[fail[u]][c];
                else{
                    fail[v]=ch[fail[u]][c];
                    val[v]+=val[fail[v]],q.pb(v);
                }
            }
        }
    }
};
```

z-algorithm.hpp

Description: Z Algorithm. z[i] := the length of the longest common prefix between s and s[i:].

8263a6, 12 lines

```
template<class STR>
vector<int> z_algorithm(const STR &s){
    int n=(int)s.size();
    vector<int> z(n);
    if(n)z[0]=n;
    for(int i=1,l=0,r=1;i<n;i++){
        if(i<r)z[i]=min(r-i,z[i-l]);
        while(i+z[i]<n&&s[z[i]]==s[i+z[i]])z[i]++;
        if(i+z[i]>r)l=i,r=i+z[i];
    }
    return z;
}
```

prefix-function.hpp

Description: Prefix function. pi[i] := the length of the longest proper prefix of s[0:i] which is also a suffix of s[0:i].

3d65fe, 11 lines

```
template<class STR>
vector<int> prefix_function(const STR &s){
    int n=(int)s.size();
    vector<int> pi(n);
    for(int i=1,j=0;i<n;i++){
        while(j>0&&s[i]!=s[j])j=pi[j-1];
        if(s[i]==s[j])j++;
        pi[i]=j;
    }
    return pi;
}
```

manacher.hpp

Description: Manacher's Algorithm. pal[i] := the length of the longest palindrome centered at i/2.

53856e, 15 lines

```
template<class STR>
vector<int> manacher(const STR &s){
    int n=(int)s.size();
    if(n==0)return {};
    vector<int> pal(2*n-1);
    for(int p=0,l=-1,r=-1;p<2*n-1;p++){
        int i=(p+1)>>1,j=p>>1;
        int k=(i>=r?0:min(r-i,pal[2*(l+r)-p]));
        while(j+k+1<n&&i-k-1>=0&&s[j+k+1]==s[i-k-1])k++;
        pal[p]=k;
        if(j+k>r)l=i-k,r=j+k;
    }
    for(int i=0;i<2*n-1;i++)pal[i]=pal[i]<<1|(i&1^1);
    return pal;
}
```

Linear Algebra (12)

12.1 Facts

12.1.1 Linear systems and rank

$A \in F^{n\times m}$, $Ax = b$ (m unknowns): solvable iff $\operatorname{rank} A = \operatorname{rank}[A \mid b]$. Solution set $= x_0 + \ker A$, $\dim \ker A = m - \operatorname{rank} A$, so over $\mathbb{F}_q$ there are $q^{m-\operatorname{rank} A}$ solutions (or 0).

- $\operatorname{rank} A = \operatorname{rank} A^T \le \min(n, m)$.
- $\operatorname{rank} AB \le \min(\operatorname{rank} A, \operatorname{rank} B)$.
- Rank-nullity: $\operatorname{rank} A + \dim \ker A = m$.
- $\operatorname{rank}(A + B) \le \operatorname{rank} A + \operatorname{rank} B$.
- Rank mod $p \le$ rank over $\mathbb{Q}$ (equal w.h.p. for random large p).

12.1.2 Determinant

- $\det AB = \det A \det B$, $\det A^T = \det A$, $\det cA = c^n \det A$.
- $\det A = 0$ iff A singular; $\det A^{-1} = 1/\det A$.
- Triangular: $\det = \prod a_{ii}$; block triangular: product of the diagonal blocks' dets.
- Row ops: swap $\to$ sign flips; scale row by $c \to \det \cdot c$; add multiple of a row $\to$ unchanged. Elimination gives $\det = (-1)^{\#\text{swaps}} \prod$ pivots (works mod p).
- $A^{-1} = \operatorname{adj}(A)/\det A$, $\operatorname{adj}(A)_{ij} = (-1)^{i+j} M_{ji}$ (M_{ji}: minor deleting row j, col i).
- Cofactor expansion: $\det A = \sum_j (-1)^{i+j} a_{ij} M_{ij}$.
- Mod composite m: run Euclid-like row elimination (swap and subtract $\lfloor a/b \rfloor$ times) instead of dividing.

12.1.3 XOR basis (GF(2))

For a multiset S of numbers with basis rank r:

- #distinct subset xors $= 2^r$.
- Each reachable value is the xor of exactly $2^{|S|-r}$ subsets; in particular $2^{|S|-r}$ subsets (incl. empty) xor to 0.
- Max xor: greedy from top bit, $x \leftarrow \max(x, x \oplus b_i)$.
- Reduced basis (each pivot bit appears only in its own vector), sorted ascending $c_0 < \cdots < c_{r-1}$: k-th smallest (0-indexed) value $= \bigoplus_{t:\, k_t=1} c_t$.
- Offline "max xor in range $[l, r]$": keep, per top bit, the vector with latest index (swap on insert); query uses only vectors with index $\geq l$.

12.1.4 Matrix-tree theorem

Undirected multigraph, Laplacian $L = D - A$ ($A_{uv} = $ #edges). #spanning trees = det of L with any row i and col i removed $= \frac{1}{n}\prod_{i=2}^{n} \lambda_i$. Weighted: $A_{uv} = \sum w$ gives $\sum_T \prod_{e \in T} w_e$. K_n: n^{n-2}; $K_{a,b}$: $a^{b-1}b^{a-1}$.

Directed: #arborescences rooted at r with edges directed away from r = det of $L_{in} = D_{in} - A$ with row/col r removed. Towards r: use $L_{out} = D_{out} - A$.

BEST theorem. Eulerian digraph (connected, in = out degree): #Eulerian circuits (as cyclic edge sequences)

$$ec(G) = t_w(G) \prod_v (\deg v - 1)!$$

where t_w = #arborescences towards any fixed w. For circuits starting at a fixed vertex s multiply by $\deg s$.

12.1.5 LGV lemma

DAG, sources $a_1..a_k$, sinks $b_1..b_k$, $e(a, b)$ = #paths (or weighted sum). With $M_{ij} = e(a_i, b_j)$,

$$\det M = \sum_{(P_1..P_k)} \operatorname{sgn}(\sigma)$$

over vertex-disjoint path tuples $P_i : a_i \to b_{\sigma(i)}$. If only $\sigma = \mathrm{id}$ can be non-intersecting (e.g. planar grid, sources/sinks in order on the boundary), $\det M$ = #non-intersecting path systems.

gauss-mod.hpp

Description: Gauss-Jordan elimination over a field $\mathbb{Z}_p$. `gauss(a,c,d)` reduces a to RREF on its first c columns (later columns are carried along), returns pivot columns and sets d = det of the left $c \times c$ block (for square use). `solve` returns rank or -1 if inconsistent, a solution x (free vars = 0) and a basis K of $\{x : Ax = 0\}$ ($|K| = m -$rank). `mat_inv` returns false if singular (then a is garbage).
Usage: `Mat<mint> A(n,vector<mint>(m)); int r=solve(A,b,m,x,K);`
Time: $\mathcal{O}(nm \cdot \min(n, m))$; 500×500 det takes 60ms.
ee701f, 54 lines

```
template<class T>
using Mat = vector<vector<T>>;

template<class T>
vector<int> gauss(Mat<T> &a,int c,T &d){
    int n=SZ(a),r=0;
    vector<int> piv;d=1;
    for(int j=0;j<c&&r<n;j++){
        int p=r;
        while(p<n&&a[p][j]==0)p++;
        if(p==n)continue;
        if(p!=r)swap(a[p],a[r]),d=-d;
        auto &ar=a[r];
        d*=ar[j];T iv=ar[j].inv();
        for(int k=j;k<SZ(ar);k++)ar[k]*=iv;
        for(int i=0;i<n;i++)if(i!=r&&!(a[i][j]==0)){
            T f=a[i][j];
            for(int k=j;k<SZ(ar);k++)a[i][k]-=f*ar[k];
        }
        piv.pb(j),r++;
    }
    if(r<c)d=0;
    return piv;
}
template<class T>
T mat_det(Mat<T> a){T d;gauss(a,SZ(a),d);return d;}
template<class T>
int mat_rank(Mat<T> a,int m){T d;return SZ(gauss(a,m,d));}
template<class T>
bool mat_inv(Mat<T> &a){
    int n=SZ(a);T d;
    for(int i=0;i<n;i++)a[i].resize(2*n),a[i][n+i]=1;
    if(SZ(gauss(a,n,d))<n)return 0;
    for(auto &r:a)r.erase(r.begin(),r.begin()+n);
    return 1;
}
template<class T>
int solve(Mat<T> a,const vector<T> &b,int m,
          vector<T> &x,Mat<T> &K){
    int n=SZ(a);T d;
    for(int i=0;i<n;i++)a[i].pb(b[i]);
    auto piv=gauss(a,m,d);
    int r=SZ(piv);
    for(int i=r;i<n;i++)if(!(a[i][m]==0))return -1;
    x=vector<T>(m),K.clear();
    vector<int> fr(m,1);
    for(int i=0;i<r;i++)x[piv[i]]=a[i][m],fr[piv[i]]=0;
    for(int j=0;j<m;j++)if(fr[j]){
        vector<T> v(m);v[j]=1;
        for(int i=0;i<r;i++)v[piv[i]]=-a[i][j];
        K.pb(v);
    }
    return r;
}
```

xor-basis.hpp

Description: Linear basis over GF(2) of 64-bit integers. `b[i]` has top bit i; `r` = rank. `ins` returns false if x was already representable (then some nonempty subset xors to 0). `mx(x)`: max of $x \oplus s$, `red(x)`: min of $x \oplus s$ over the span s (so x is representable iff `red(x)==0`). `mn()`: min nonzero span value. `kth(k)`: k-th smallest (0-indexed, $k < 2^r$, 0 counts) span value. `gauss_xor`: solves $Ax = b$ over GF(2), row i = bits $0..m-1$ of `a[i]`, b_i = bit m. Returns rank or -1; free variables = 0.
Time: $\mathcal{O}(64)$ per op, `kth` $\mathcal{O}\left(64^2\right)$; `gauss_xor` $\mathcal{O}\left(nm \cdot \min(n, m)/64\right)$
63444c, 47 lines

```
struct XorBasis{
    u64 b[64]={};int r=0;
    bool ins(u64 x){
        for(int i=63;i>=0;i--)if(x>>i&1){
            if(!b[i])return b[i]=x,++r;
            x^=b[i];
        }
        return 0;
    }
    u64 red(u64 x){
        for(int i=63;i>=0;i--)chmin(x,x^b[i]);
        return x;
    }
    u64 mx(u64 x=0){
        for(int i=63;i>=0;i--)chmax(x,x^b[i]);
        return x;
    }
    u64 mn(){
        for(int i=0;i<64;i++)if(b[i])return b[i];
        return 0;
    }
    u64 kth(u64 k){
        u64 res=0;int t=0;
        for(int i=0;i<64;i++)if(b[i]){
            for(int j=i-1;j>=0;j--)if(b[i]>>j&1)b[i]^=b[j];
            if(k>>t++&1)res^=b[i];
        }
        return res;
    }
};

template<size_t N>
int gauss_xor(vector<bitset<N>> a,int m,bitset<N> &x){
    int n=SZ(a),r=0;vector<int> piv;
    for(int j=0;j<m&&r<n;j++){
        int p=r;
        while(p<n&&!a[p][j])p++;
        if(p==n)continue;
        swap(a[p],a[r]);
        for(int i=0;i<n;i++)if(i!=r&&a[i][j])a[i]^=a[r];
        piv.pb(j),r++;
    }
    for(int i=r;i<n;i++)if(a[i][m])return -1;
    x.reset();
    for(int i=0;i<r;i++)x[piv[i]]=a[i][m];
    return r;
}
```

Linear Programming (13)

13.1 LP Duality

primal problem:

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{subject to} \quad & Ax \leq b \\ & x \geq 0 \end{aligned}$$

dual problem:

$$\begin{aligned} \text{minimize} \quad & b^T y \\ \text{subject to} \quad & A^T y \geq c \\ & y \geq 0 \end{aligned}$$

Maximization	Minimization
Inequality constraint $\le$	Nonnegative variable $\ge$
Inequality constraint $\ge$	Nonpositive variable $\le$
Equality constraint $=$	Free variable
Nonnegative variable $\ge$	Inequality constraint $\ge$
Nonpositive variable $\le$	Inequality constraint $\le$
Free variable	Equality constraint $=$

Complementary slackness: x, y optimal iff feasible and $x_j > 0 \Rightarrow$ dual constraint j is tight, $y_i > 0 \Rightarrow$ primal constraint i is tight.

13.2 Well-known Duals

13.2.1 Shortest path / difference constraints

Unit flow $s \to t$, $f \ge 0$: $\min \sum w_e f_e \Leftrightarrow$

$$\max\ \pi_t - \pi_s \quad \text{s.t. } \pi_v - \pi_u \le w_{uv}\ \forall (u,v)$$

Optimum $\pi_v = \text{dist}(s, v)$. A system $x_v - x_u \le w_{uv}$ is feasible iff the graph has no negative cycle; a solution is $x_v = \text{dist}$ from a virtual source with 0-edges to all nodes (Bellman-Ford), and it maximizes $\sum x_v$ subject to $x_v \le 0$. Dijkstra potentials (Johnson) are dual feasible: $w_{uv} + h_u - h_v \ge 0$.

13.2.2 Max flow / min cut

max flow $\Leftrightarrow \min \sum c_e z_e$ s.t. $z_{uv} \ge \pi_u - \pi_v$, $\pi_s - \pi_t \ge 1$, $z \ge 0$; an optimal $\pi \in \{0, 1\}$ is a min cut ($\pi = 1$ on the s side).

13.2.3 Min cost flow

Supplies $b_v = \text{out} - \text{in}$, $0 \le f_e \le u_e$, cost c_e, $\sum b_v = 0$:

$$\min \sum_e c_e f_e = \max_\pi \sum_v b_v \pi_v - \sum_{e=(u,v)} u_e \max(0, \pi_u - \pi_v - c_e)$$

(π free; integral optimum for integral data; dual unbounded iff no feasible flow). **Contest use** (read right to left): to choose integers x_v minimizing

$$\sum_{e=(u,v)} u_e \max(0, x_u - x_v - c_e) - \sum_v b_v x_v,$$

the answer is $-$(min cost flow) with supplies b_v and edges $u \to v$ (cap u_e, cost c_e). Hard constraint $x_u - x_v \le c$: cap ∞. To use constants / bounds ($l \le x_v \le r$) add a node 0 with $x_0 = 0$ and give it supply $-\sum b_v$. Convex piecewise-linear costs: a sum of such max terms.

13.2.4 Assignment

$\max \sum w_{ij} x_{ij}$ (perfect matching) $\Leftrightarrow \min \sum_i u_i + \sum_j v_j$ s.t. $u_i + v_j \ge w_{ij}$ (Hungarian potentials). Unweighted bipartite: max matching = min vertex cover (König).

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \le b$, $x \ge 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.
Usage: `vvd A = {{1,-1}, {-1,1}, {-1,-2}};`
`vd b = {1,1,-4}, c = {-1,-1}, x;`
`T val = LPSolver(A, b, c).solve(x);`
Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case.
d643f3, 68 lines

```
typedef double T; // long double, Rational, double + mod<P>...
typedef vector<T> vd;
typedef vector<vd> vvd;

const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j

struct LPSolver {
  int m, n;
  vector<int> N, B;
  vvd D;

  LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(SZ(b)), n(SZ(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
      for(int i=0;i<m;i++) for(int j=0;j<n;j++) D[i][j] = A[i][
          j];
      for(int i=0;i<m;i++) { B[i] = n+i; D[i][n] = -1; D[i][n
          +1] = b[i];}
      for(int j=0;j<n;j++) { N[j] = j; D[m][j] = -c[j]; }
      N[n] = -1; D[m+1][n] = 1;
    }

  void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    for(int i=0;i<m+2;i++) if (i != r && abs(D[i][s]) > eps) {
      T *b = D[i].data(), inv2 = b[s] * inv;
      for(int j=0;j<n+2;j++) b[j] -= a[j] * inv2;
      b[s] = a[s] * inv2;
    }
    for(int j=0;j<n+2;j++) if (j != s) D[r][j] *= inv;
    for(int i=0;i<m+2;i++) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
  }

  bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
      int s = -1;
      for(int j=0;j<n+1;j++) if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return true;
      int r = -1;
      for(int i=0;i<m;i++) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                     < MP(D[r][n+1] / D[r][s], B[r])) r = i;
      }
      if (r == -1) return false;
      pivot(r, s);
    }
  }

  T solve(vd &x) {
    int r = 0;
    for(int i=1;i<m;i++) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
      pivot(r, n);
      if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
      for(int i=0;i<m;i++) if (B[i] == -1) {
        int s = 0;
        for(int j=1;j<n+1;j++) ltj(D[i]);
        pivot(i, s);
      }
    }
    bool ok = simplex(1); x = vd(n);
    for(int i=0;i<m;i++) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
  }
};
```

Geometry (14)

14.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)
9c840e, 28 lines

```
template<class T> int sgn(T x){return (x>0)-(x<0);}
template<class T>
struct Point{
  using P = Point;
  T x,y;
  explicit Point(T x=0,T y=0):x(x),y(y){}
  bool operator<(P p)const{return tie(x,y)<tie(p.x,p.y);}
  bool operator==(P p)const{return tie(x,y)==tie(p.x,p.y);}
  P operator+(P p)const{return P(x+p.x,y+p.y);}
  P operator-(P p)const{return P(x-p.x,y-p.y);}
  P operator*(T d)const{return P(x*d,y*d);}
  P operator/(T d)const{return P(x/d,y/d);}
  T dot(P p)const{return x*p.x+y*p.y;}
  T cross(P p)const{return x*p.y-y*p.x;}
  T cross(P a,P b)const{return (a-*this).cross(b-*this);}
  T dist2()const{return x*x+y*y;}
  double dist()const{return sqrt((double)dist2());}
  // angle to x-axis in interval [-pi, pi]
  double angle()const{return atan2(y,x);}
  P unit()const{return *this/dist();} // makes dist()=1
  P perp()const{return P(-y,x);} // rotates +90 degrees
  P normal()const{return perp().unit();}
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a)const{
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a));}
  friend ostream& operator<<(ostream& os,P p){
    return os<<"("<<p.x<<","<<p.y<<")";}
};
```

GeoUtils.h

Description: Basic primitives: lineDist, sideOf, onSegment, segDist, lineInter, segInter, lineProj, linearTransformation, ccRadius/ccCenter (circumcircle), polygonArea2, polygonCenter. Integer points: products of up to three coordinates, watch overflow.
Time: $\mathcal{O}(1)$, polygon ones $\mathcal{O}(n)$
"src/geometry/Point.h"
4cfb4a, 81 lines

```
// signed dist from p to line ab (>0 left); a==b gives nan
template<class P> double lineDist(P a,P b,P p){
  return (double)(b-a).cross(p-a)/(b-a).dist();
}
// p seen from s towards e: 1 left, 0 on, -1 right
template<class P>
int sideOf(P s,P e,P p){return sgn(s.cross(e,p));}
// eps version: 0 if p within distance eps of line
template<class P> int sideOf(P s,P e,P p,double eps){
  auto a=(e-s).cross(p-s);
```

```
  double l=(e-s).dist()*eps;
  return (a>l)-(a<-l);
}
// p on segment se (exact; for doubles use segDist<=eps)
template<class P> bool onSegment(P s,P e,P p){
  return p.cross(s,e)==0&&(s-p).dot(e-p)<=0;
}
// dist from p to segment se (Point<double>)
template<class P> double segDist(P s,P e,P p){
  if(s==e)return (p-s).dist();
  auto d=(e-s).dist2(),t=min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
// lines s1e1,s2e2: {1,p} unique, {0,(0,0)} parallel,
// {-1,(0,0)} same line. Wrong if Point<ll> and p not integer
template<class P> pair<int,P> lineInter(P s1,P e1,P s2,P e2){
  auto d=(e1-s1).cross(e2-s2);
  if(d==0)return {-(s1.cross(e1,s2)==0),P(0,0)}; // parallel
  auto p=s2.cross(e1,e2),q=s2.cross(e2,s1);
  return {1,(s1*p+e1*q)/d};
}
// segments ab,cd: {} none, {p} one point, {p,q} overlap
template<class P> vector<P> segInter(P a,P b,P c,P d){
  auto oa=c.cross(d,a),ob=c.cross(d,b);
  auto oc=a.cross(b,c),od=a.cross(b,d);
  if(sgn(oa)*sgn(ob)<0&&sgn(oc)*sgn(od)<0)
    return {(a*ob-b*oa)/(ob-oa)};
  set<P> s;
  if(onSegment(c,d,a))s.insert(a);
  if(onSegment(c,d,b))s.insert(b);
  if(onSegment(a,b,c))s.insert(c);
  if(onSegment(a,b,d))s.insert(d);
  return {ALL(s)};
}
// projection of p onto line ab (refl: reflection instead)
template<class P> P lineProj(P a,P b,P p,bool refl=false){
  P v=b-a;
  return p-v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
// move the plane (rotate+scale+translate, never mirror) so
// p0->q0 and p1->q1; returns where r lands. As complex
// numbers: q0+(r-p0)*(q1-q0)/(p1-p0). Use Point<double>
template<class P>
P linearTransformation(P p0,P p1,P q0,P q1,P r){
  P dp=p1-p0,dq=q1-q0,num(dp.cross(dq),dp.dot(dq));
  return q0+P((r-p0).cross(num),(r-p0).dot(num))/dp.dist2();
}
// circumcircle of ABC: radius / center (Point<double>)
template<class P> double ccRadius(P A,P B,P C){
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
    abs((B-A).cross(C-A))/2;
}
template<class P> P ccCenter(const P& A,const P& B,const P& C){
  P b=C-A,c=B-A;
  return A+(b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
// twice signed area (CW negative)
template<class T> T polygonArea2(const vector<Point<T>>& v){
  T a=v.back().cross(v[0]);
  for(int i=0;i<SZ(v)-1;i++)a+=v[i].cross(v[i+1]);
  return a;
}
// center of mass of polygon (Point<double>)
template<class P> P polygonCenter(const vector<P>& v){
  P res(0,0);double A=0;
  for(int i=0,j=SZ(v)-1;i<SZ(v);j=i++){
    res=res+(v[i]+v[j])*v[j].cross(v[i]);
    A+=v[j].cross(v[i]);
  }
  return res/A/3;
}
```

14.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"src/geometry/Point.h" 93169e, 10 lines

```
using P = Point<double>;
bool circleInter(P a,P b,double r1,double r2,pair<P,P>* out){
  if(a==b){assert(r1!=r2);return false;}
  P vec=b-a;
  double d2=vec.dist2(),sum=r1+r2,dif=r1-r2,
    p=(d2+r1*r1-r2*r2)/(d2*2),h2=r1*r1-p*p*d2;
  if(sum*sum<d2||dif*dif>d2)return false;
  P mid=a+vec*p,per=vec.perp()*sqrt(fmax(0,h2)/d2);
  *out={mid+per,mid-per};return true;
}
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"src/geometry/Point.h" b0153d, 13 lines

```
template<class P>
vector<pair<P,P>> tangents(P c1,double r1,P c2,double r2){
  P d=c2-c1;
  double dr=r1-r2,d2=d.dist2(),h2=d2-dr*dr;
  if(d2==0||h2<0)return {};
  vector<pair<P,P>> out;
  for(double sign:{-1,1}){
    P v=(d*dr+d.perp()*sqrt(h2)*sign)/d2;
    out.push_back({c1+v*r1,c2+v*r2});
  }
  if(h2==0)out.pop_back();
  return out;
}
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

"src/geometry/Point.h" e0cfba, 8 lines

```
template<class P> vector<P> circleLine(P c,double r,P a,P b){
  P ab=b-a,p=a+ab*(c-a).dot(ab)/ab.dist2();
  double s=a.cross(b,c),h2=r*r-s*s/ab.dist2();
  if(h2<0)return {};
  if(h2==0)return {p};
  P h=ab.unit()*sqrt(h2);
  return {p-h,p+h};
}
```

CirclePolyInter.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

Time: $\mathcal{O}(n)$

"src/geometry/Point.h" 0e23a5, 18 lines

```
using P = Point<double>;
#define ARG(p,q) atan2(p.cross(q),p.dot(q))
double circlePoly(P c,double r,vector<P> ps){
  auto tri=[&](P p,P q){
    auto r2=r*r/2;P d=q-p;
    auto a=d.dot(p)/d.dist2(),b=(p.dist2()-r*r)/d.dist2();
    auto det=a*a-b;
    if(det<=0)return ARG(p,q)*r2;
    auto s=max(0.,-a-sqrt(det)),t=min(1.,-a+sqrt(det));
    if(t<0||1<=s)return ARG(p,q)*r2;
    P u=p+d*s,v=q+d*(t-1);
    return ARG(p,u)*r2+u.cross(v)/2+ARG(v,q)*r2;
  };
  auto sum=0.0;
  for(int i=0;i<SZ(ps);i++)
    sum+=tri(ps[i]-c,ps[(i+1)%SZ(ps)]-c);
  return sum;
}
```

MinEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.

Time: expected $\mathcal{O}(n)$

"src/geometry/GeoUtils.h" aa8834, 15 lines

```
using P = Point<double>;

pair<P,double> mec(vector<P> ps){
  shuffle(ALL(ps),mt19937(time(0)));
  P o=ps[0];double r=0,EPS=1+1e-8;
  for(int i=0;i<SZ(ps);i++)if((o-ps[i]).dist()>r*EPS){
    o=ps[i],r=0;
    for(int j=0;j<i;j++)if((o-ps[j]).dist()>r*EPS){
      o=(ps[i]+ps[j])/2,r=(o-ps[i]).dist();
      for(int k=0;k<j;k++)if((o-ps[k]).dist()>r*EPS)
        o=ccCenter(ps[i],ps[j],ps[k]),r=(o-ps[i]).dist();
    }
  }
  return {o,r};
}
```

14.3 Polygons

InsidePoly.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

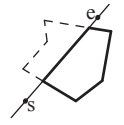
Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`

Time: $\mathcal{O}(n)$

"src/geometry/Point.h", "src/geometry/GeoUtils.h" 42e1d3, 11 lines

```
template<class P>
bool inPolygon(vector<P>& p,P a,bool strict=true){
  int cnt=0,n=SZ(p);
  for(int i=0;i<n;i++){
    P q=p[(i+1)%n];
    if(onSegment(p[i],q,a))return !strict;
    //or: if(segDist(p[i],q,a)<=eps)return !strict;
    cnt^=((a.y<p[i].y)-(a.y<q.y))*a.cross(p[i],q)>0;
  }
  return cnt;
}
```

PolyCut.h



Description:
Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.

Usage: `vector<P> p = ...;`
`p = polygonCut(p, P(0,0), P(1,0));`

"src/geometry/Point.h" a29a00, 11 lines

```
using P = Point<double>;
vector<P> polygonCut(const vector<P>& poly,P s,P e){
  vector<P> res;
  for(int i=0;i<SZ(poly);i++){
    P cur=poly[i],prev=i?poly[i-1]:poly.back();
```

```
    auto a=s.cross(e,cur),b=s.cross(e,prev);
    if((a<0)!=(b<0))res.push_back(cur+(prev-cur)*(a/(a-b)));
    if(a<0)res.push_back(cur);
  }
  return res;
}
```

HalfPlane.h

Description: Intersection of half-planes. Half-plane (s, e) keeps the points LEFT of the directed line $s \to e$, i.e. $s.cross(e, p) \geq 0$ (opposite of polygonCut, which keeps the right side). Returns the vertices in ccw order, empty if the intersection is empty or has zero area. The result must be bounded: add a bounding box. EPS is absolute: with a huge box ($B > 10^5$) zero-area answers may survive as thin slivers.
Usage: `vector<L> h={...}; double B=1e5;`
`P c[4]={P(-B,-B),P(B,-B),P(B,B),P(-B,B)};`
`for(int i=0;i<4;i++)h.pb({c[i],c[(i+1)vector<P>`
`poly=halfPlane(h);`
Time: $\mathcal{O}(n \log n)$

"src/geometry/Point.h" d17db1, 37 lines

```
using P = Point<double>;
using L = array<P,2>;
P hpInter(L a,L b){
  P d=a[1]-a[0],e=b[1]-b[0];
  return a[0]+d*((b[0]-a[0]).cross(e)/d.cross(e));
}
vector<P> halfPlane(vector<L> h){
  auto dir=[](L l){return l[1]-l[0];};
  auto out=[](L l,P p){return l[0].cross(l[1],p)<-EPS;};
  auto hf=[&](L l){P d=dir(l);return d.y<0||(!d.y&&d.x<0);};
  auto cr=[&](L a,L b){return dir(a).cross(dir(b));};
  sort(ALL(h),[&](L a,L b){
    if(hf(a)!=hf(b))return hf(a)<hf(b);
    return cr(a,b)?cr(a,b)>0:b[0].cross(b[1],a[0])>0;});
  deque<L> q;
  auto bk=[&]{return hpInter(q[SZ(q)-1],q[SZ(q)-2]);};
  auto fr=[&]{return hpInter(q[0],q[1]);};
  for(int i=0;i<SZ(h);i++){
    L l=h[i];
    if(i&&hf(l)==hf(h[i-1])&&!cr(l,h[i-1]))continue;
    while(SZ(q)>1&&out(l,bk()))q.pop_back();
    while(SZ(q)>1&&out(l,fr()))q.pop_front();
    if(SZ(q)&&abs(dir(l).cross(dir(q.back())))<EPS){
      if(dir(l).dot(dir(q.back()))<0)return {};
      if(out(l,q.back()[0]))q.pop_back();
      else continue;
    }
    q.pb(l);
  }
  while(SZ(q)>2&&out(q[0],bk()))q.pop_back();
  while(SZ(q)>2&&out(q.back(),fr()))q.pop_front();
  if(SZ(q)<3)return {};
  vector<P> r;double a=0;
  for(int i=0;i<SZ(q);i++)r.pb(hpInter(q[i],q[(i+1)%SZ(q)]));
  for(int i=0;i<SZ(r);i++)a+=r[i].cross(r[(i+1)%SZ(r)]);
  return a>EPS?r:vector<P>();
}
```

PolyUnion.h

Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)
Time: $\mathcal{O}\left(N^2\right)$, where N is the total number of points

"src/geometry/Point.h", "src/geometry/GeoUtils.h" 28b401, 32 lines

```
using P = Point<double>;
double rat(P a,P b){return sgn(b.x)?a.x/b.x:a.y/b.y;}
double polyUnion(vector<vector<P>>& poly){
  double ret=0;
  for(int i=0;i<SZ(poly);i++)for(int v=0;v<SZ(poly[i]);v++){
    P A=poly[i][v],B=poly[i][(v+1)%SZ(poly[i])];
    vector<pair<double,int>> segs={{0,0},{1,0}};
    for(int j=0;j<SZ(poly);j++)if(i!=j){
      for(int u=0;u<SZ(poly[j]);u++){
        P C=poly[j][u],D=poly[j][(u+1)%SZ(poly[j])];
        int sc=sideOf(A,B,C),sd=sideOf(A,B,D);
        if(sc!=sd){
          double sa=C.cross(D,A),sb=C.cross(D,B);
          if(min(sc,sd)<0)
            segs.emplace_back(sa/(sa-sb),sgn(sc-sd));
        }else if(!sc&&!sd&&j<i&&sgn((B-A).dot(D-C))>0){
          segs.emplace_back(rat(C-A,B-A),1);
          segs.emplace_back(rat(D-A,B-A),-1);
        }
      }
    }
    sort(ALL(segs));
    for(auto &s:segs)s.first=min(max(s.first,0.0),1.0);
    double sum=0;int cnt=segs[0].second;
    for(int j=1;j<SZ(segs);j++){
      if(!cnt)sum+=segs[j].first-segs[j-1].first;
      cnt+=segs[j].second;
    }
    ret+=A.cross(B)*sum;
  }
  return ret/2;
}
```

ConvexHull.h

Description:
Returns a vector of the points of the convex hull in counterclockwise order. Points on the edge of the hull between two other points are not considered part of the hull.



Time: $\mathcal{O}(n \log n)$

"src/geometry/Point.h" f3887b, 12 lines

```
using P = Point<ll>;
vector<P> convexHull(vector<P> pts){
  if(SZ(pts)<=1)return pts;
  sort(ALL(pts));
  vector<P> h(SZ(pts)+1);int s=0,t=0;
  for(int it=2;it--;s=--t,reverse(ALL(pts)))
    for(P p:pts){
      while(t>=s+2&&h[t-2].cross(h[t-1],p)<=0)t--;
      h[t++]=p;
    }
  return {h.begin(),h.begin()+t-(t==2&&h[0]==h[1])};
}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).
Time: $\mathcal{O}(n)$

"src/geometry/Point.h" fd71ab, 11 lines

```
using P = Point<ll>;
array<P,2> hullDiameter(vector<P> S){
  int n=SZ(S),j=n<2?0:1;
  pair<ll,array<P,2>> res({0,{S[0],S[0]}});
  for(int i=0;i<j;i++)
    for(;;j=(j+1)%n){
      res=max(res,{(S[i]-S[j]).dist2(),{S[i],S[j]}});
      if((S[(j+1)%n]-S[j]).cross(S[i+1]-S[i])>=0)break;
    }
  return res.second;
}
```

MaxTriangle.h

Description: Returns twice the maximum area of a triangle with vertices among the given points, and the triangle (0 if $n < 3$ or all collinear). On the convex hull, the best triangle rooted at r (vertices $r < j < k$) interleaves with those rooted at $a < r < b$, so divide and conquer over roots, each rooted search being a two-pointer scan of the allowed ranges.
Usage: `auto [a2,t]=maxTriangle(pts);`
Time: $\mathcal{O}(n \log n)$

"src/geometry/ConvexHull.h" 920b41, 32 lines

```
pair<ll,array<P,3>> maxTriangle(vector<P> pts){
  vector<P> h=convexHull(pts);
  int n=SZ(h);
  pair<ll,array<P,3>> res{};
  if(n<3)return res;
  using J = array<int,2>;
  auto at=[&](int i){return h[i%n];};
  auto A=[&](int a,int b,int c){
    return at(a).cross(at(b),at(c));};
  // best (r,j,k) with j in [jl,jr], k in [kl,kr], j<k
  auto rooted=[&](int r,int jl,int jr,int kl,int kr){
    J t{jl,kl};ll mx=-1;
    for(int j=jl,k=kl;j<=jr;j++){
      k=max(k,j+1);
      if(k>kr)break;
      while(k<kr&&A(r,j,k+1)>=A(r,j,k))k++;
      if(A(r,j,k)>mx)mx=A(r,j,k),t={j,k};
    }
    res=max(res,{mx,{at(r),at(t[0]),at(t[1])}});
    return t;
  };
  auto rec=[&](auto &&self,int a,int b,J L,J R)->void {
    if(b-a<2)return;
    int m=(a+b)/2;
    J M=rooted(m,max(L[0],m+1),min(R[0],m+n-2),
      max(L[1],m+2),min(R[1],m+n-1));
    self(self,a,m,L,M),self(self,m,b,M,R);
  };
  J t=rooted(0,1,n-2,2,n-1);
  rec(rec,0,n,t,{t[0]+n,t[1]+n});
  return res;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.
Time: $\mathcal{O}(\log N)$

"src/geometry/Point.h", "src/geometry/GeoUtils.h" 7d714c, 13 lines

```
using P = Point<ll>;
bool inHull(const vector<P>& l,P p,bool strict=true){
  int a=1,b=SZ(l)-1,r=!strict;
  if(SZ(l)<3)return r&&onSegment(l[0],l.back(),p);
  if(sideOf(l[0],l[a],l[b])>0)swap(a,b);
  if(sideOf(l[0],l[a],p)>=r||sideOf(l[0],l[b],p)<=-r)
    return false;
  while(abs(a-b)>1){
    int c=(a+b)/2;
    (sideOf(l[0],l[c],p)>0?b:a)=c;
  }
  return sgn(l[a].cross(l[b],p))<r;
}
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: • $(-1,-1)$ if no collision, • $(i,-1)$ if touching the corner i, • (i,i) if along side $(i,i+1)$, • (i,j) if crossing sides $(i,i+1)$ and $(j,j+1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i,i+1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.
Time: $\mathcal{O}(\log n)$

"src/geometry/Point.h" 4cc60f, 36 lines

```
#define CMP(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define EXTR(i) CMP(i+1,i)>=0&&CMP(i,i-1+n)<0
template<class P> int extrVertex(vector<P>& poly,P dir){
  int n=SZ(poly),lo=0,hi=n;
  if(EXTR(0))return 0;
  while(lo+1<hi){
    int m=(lo+hi)/2;if(EXTR(m))return m;
    int ls=CMP(lo+1,lo),ms=CMP(m+1,m);
    (ls<ms||(ls==ms&&ls==CMP(lo,m))?hi:lo)=m;
  }
  return lo;
}
#define CMPL(i) sgn(a.cross(poly[i],b))
template<class P>
array<int,2> lineHull(P a,P b,vector<P>& poly){
  int endA=extrVertex(poly,(a-b).perp());
  int endB=extrVertex(poly,(b-a).perp());
  if(CMPL(endA)<0||CMPL(endB)>0)return {-1,-1};
  array<int,2> res;
  for(int i=0;i<2;i++){
    int lo=endB,hi=endA,n=SZ(poly);
    while((lo+1)%n!=hi){
      int m=((lo+hi+(lo<hi?0:n))/2)%n;
      (CMPL(m)==CMPL(endB)?lo:hi)=m;
    }
    res[i]=(lo+!CMPL(hi))%n;
    swap(endA,endB);
  }
  if(res[0]==res[1])return {res[0],-1};
  if(!CMPL(res[0])&&!CMPL(res[1]))
    switch((res[0]-res[1]+SZ(poly)+1)%SZ(poly)){
      case 0:return {res[0],res[0]};
      case 2:return {res[1],res[1]};
    }
  return res;
}
```

14.3.1 Pick's Theorem

Given a simple polygon with vertices on an integer grid:

$$A = I + \frac{B}{2} - 1 \implies I = A - \frac{B}{2} + 1$$

Where A is the area, I is the number of strictly interior lattice points, and B is the number of boundary lattice points.

- $2A = \left|\sum_{i=0}^{n-1}(x_i y_{i+1} - x_{i+1}y_i)\right|$ (Shoelace), indices mod n; the signed sum is positive iff the vertices are ccw.
- $B = \sum \gcd(|x_{i+1}-x_i|, |y_{i+1}-y_i|)$.
- In integers: $I = (2A - B + 2)/2$.

14.4 Misc. Point Set Problems

ManhattanMST.h

Description: Given N points, returns up to 4*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights w(p, q) = —p.x - q.x— + —p.y - q.y—. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.
Time: $\mathcal{O}(N \log N)$

"src/geometry/Point.h" 321d39, 21 lines

```
using P = Point<ll>;
vector<array<ll,3>> manhattanMST(vector<P> ps){
  vector<int> id(SZ(ps));iota(ALL(id),0);
  vector<array<ll,3>> edges;
  for(int k=0;k<4;k++){
    sort(ALL(id),[&](int i,int j){
      return (ps[i]-ps[j]).x<(ps[j]-ps[i]).y;});
    map<ll,int> sweep;
    for(int i:id){
      for(auto it=sweep.lower_bound(-ps[i].y);
          it!=sweep.end();sweep.erase(it++)){
        int j=it->second;P d=ps[i]-ps[j];
        if(d.y>d.x)break;
        edges.push_back({d.y+d.x,i,j});
      }
      sweep[-ps[i].y]=i;
    }
    for(P &p:ps)if(k&1)p.x=-p.x;else swap(p.x,p.y);
  }
  return edges;
}
```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

"src/geometry/Point.h" c243f7, 54 lines

```
using T = ll;
using P = Point<T>;
const T KINF=numeric_limits<T>::max();

bool on_x(const P& a,const P& b){return a.x<b.x;}
bool on_y(const P& a,const P& b){return a.y<b.y;}

struct Node{
  P pt; // if this is a leaf, the single point in it
  T x0=KINF,x1=-KINF,y0=KINF,y1=-KINF; // bounds
  Node *first=0,*second=0;
  T distance(const P& p){ // min squared distance to a point
    T x=(p.x<x0?x0:p.x>x1?x1:p.x);
    T y=(p.y<y0?y0:p.y>y1?y1:p.y);
    return (P(x,y)-p).dist2();
  }
  Node(vector<P>&& vp):pt(vp[0]){
    for(P p:vp){
      x0=min(x0,p.x);x1=max(x1,p.x);
      y0=min(y0,p.y);y1=max(y1,p.y);
    }
    if(vp.size()>1){
      // split on x if width >= height (not ideal...)
      sort(ALL(vp),x1-x0>=y1-y0?on_x:on_y);
      // give half the array to each child (not best
      // performance with many duplicates in the middle)
      int half=SZ(vp)/2;
      first=new Node({vp.begin(),vp.begin()+half});
      second=new Node({vp.begin()+half,vp.end()});
    }
  }
};

struct KDTree{
  Node* root;
  KDTree(const vector<P>& vp):root(new Node({ALL(vp)})){}
  pair<T,P> search(Node *node,const P& p){
    if(!node->first){
      // uncomment if we should not find the point itself:
      // if(p==node->pt)return {KINF,P()};
      return make_pair((p-node->pt).dist2(),node->pt);
    }
    Node *f=node->first,*s=node->second;
    T bfirst=f->distance(p),bsec=s->distance(p);
    if(bfirst>bsec)swap(bsec,bfirst),swap(f,s);
    // search closest side first, other side if needed
    auto best=search(f,p);
    if(bsec<best.first)best=min(best,search(s,p));
    return best;
  }
  // find nearest point to a point, and its squared distance
  // (requires an arbitrary operator< for Point)
  pair<T,P> nearest(const P& p){return search(root,p);}
};
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.
Time: $\mathcal{O}(n \log n)$

"src/geometry/Point.h" a1d7b1, 80 lines

```
using P = Point<ll>;
using Q = struct Quad*;
using lll = __int128_t; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point

struct Quad{
  Q rot,o;P p=arb;bool mark;
  P& F(){return r()->p;}
  Q& r(){return rot->rot;}
  Q prev(){return rot->o->rot;}
  Q next(){return r()->prev();}
}*H;

bool circ(P p,P a,P b,P c){ // is p in the circumcircle?
  lll p2=p.dist2(),A=a.dist2()-p2,
      B=b.dist2()-p2,C=c.dist2()-p2;
  return p.cross(a,b)*C+p.cross(b,c)*A+p.cross(c,a)*B>0;
}
Q makeEdge(P orig,P dest){
  Q r=H?H:new Quad{new Quad{new Quad{new Quad{0}}}};
  H=r->o;r->r()->r()=r;
  for(int i=0;i<4;i++)r=r->rot,r->p=arb,r->o=i&1?r:r->r();
  r->p=orig;r->F()=dest;
  return r;
}
void splice(Q a,Q b){
  swap(a->o->rot->o,b->o->rot->o);swap(a->o,b->o);
}
Q connect(Q a,Q b){
  Q q=makeEdge(a->F(),b->p);
  splice(q,a->next());splice(q->r(),b);
  return q;
}

pair<Q,Q> rec(const vector<P>& s){
  if(SZ(s)<=3){
    Q a=makeEdge(s[0],s[1]),b=makeEdge(s[1],s.back());
    if(SZ(s)==2)return {a,a->r()};
    splice(a->r(),b);
    auto side=s[0].cross(s[1],s[2]);
    Q c=side?connect(b,a):0;
    return {side<0?c->r():a,side<0?c:b->r()};
  }
```

```
#define H(e) e->F(),e->p
#define VALID(e) (e->F().cross(H(base))>0)
  Q A,B,ra,rb;
  int half=SZ(s)/2;
  tie(ra,A)=rec({ALL(s)-half});
  tie(B,rb)=rec({SZ(s)-half+ALL(s)});
  while((B->p.cross(H(A))<0&&(A=A->next()))||
        (A->p.cross(H(B))>0&&(B=B->r()->o)));
  Q base=connect(B->r(),A);
  if(A->p==ra->p)ra=base->r();
  if(B->p==rb->p)rb=base;
#define DEL(e,init,dir) Q e=init->dir;if(VALID(e)) \
  while(circ(e->dir->F(),H(base),e->F())){Q t=e->dir; \
    splice(e,e->prev());splice(e->r(),e->r()->prev()); \
    e->o=H;H=e;e=t;}
  for(;;){
    DEL(LC,base->r(),o);DEL(RC,base,prev());
    if(!VALID(LC)&&!VALID(RC))break;
    if(!VALID(LC)||(VALID(RC)&&circ(H(RC),H(LC))))
      base=connect(RC,base->r());
    else base=connect(base->r(),LC->r());
  }
  return {ra,rb};
}

vector<P> triangulate(vector<P> pts){
  sort(ALL(pts));assert(unique(ALL(pts))==pts.end());
  if(SZ(pts)<2)return {};
  Q e=rec(pts).first;
  vector<Q> q={e};int qi=0;
  while(e->o->F().cross(e->F(),e->p)<0)e=e->o;
#define ADD {Q c=e;do{c->mark=1;pts.push_back(c->p); \
  q.push_back(c->r());c=c->next();}while(c!=e);}
  ADD;pts.clear();
  while(qi<SZ(q))if(!(e=q[qi++])->mark)ADD;
  return pts;
}
```

14.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

3058c3, 6 lines

```
template<class V,class L>
double signedPolyVolume(const V& p,const L& trilist){
  double v=0;
  for(auto i:trilist)v+=p[i.a].cross(p[i.b]).dot(p[i.c]);
  return v/6;
}
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

e0e4cf, 30 lines

```
template<class T> struct Point3D{
  using P = Point3D;
  using R = const P&;
  T x,y,z;
  explicit Point3D(T x=0,T y=0,T z=0):x(x),y(y),z(z){}
  bool operator<(R p)const{return tie(x,y,z)<tie(p.x,p.y,p.z);}
  bool operator==(R p)const{
    return tie(x,y,z)==tie(p.x,p.y,p.z);}
  P operator+(R p)const{return P(x+p.x,y+p.y,z+p.z);}
  P operator-(R p)const{return P(x-p.x,y-p.y,z-p.z);}
  P operator*(T d)const{return P(x*d,y*d,z*d);}
  P operator/(T d)const{return P(x/d,y/d,z/d);}
  T dot(R p)const{return x*p.x+y*p.y+z*p.z;}
  P cross(R p)const{
    return P(y*p.z-z*p.y,z*p.x-x*p.z,x*p.y-y*p.x);}
  T dist2()const{return x*x+y*y+z*z;}
  double dist()const{return sqrt((double)dist2());}
  // azimuthal angle (longitude) to x-axis in [-pi,pi]
  double phi()const{return atan2(y,x);}
  // zenith angle (latitude) to the z-axis in [0,pi]
  double theta()const{return atan2(sqrt(x*x+y*y),z);}
  P unit()const{return *this/(T)dist();} // makes dist()=1
  // unit vector normal to *this and p
  P normal(P p)const{return cross(p).unit();}
  // point rotated by angle (radians) ccw around axis
  P rotate(double angle,P axis)const{
    double s=sin(angle),c=cos(angle);P u=axis.unit();
    return u*dot(u)*(1-c)+(*this)*c-cross(u)*s;
  }
};
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Time: $\mathcal{O}\left(n^2\right)$

"Point3D.h" 61c74a, 44 lines

```
using P3 = Point3D<double>;

struct PR{
  void ins(int x){(a==-1?a:b)=x;}
  void rem(int x){(a==x?a:b)=-1;}
  int cnt(){return (a!=-1)+(b!=-1);}
  int a,b;
};

struct F{P3 q;int a,b,c;};

vector<F> hull3d(const vector<P3>& A){
  assert(SZ(A)>=4);
  vector<vector<PR>> E(SZ(A),vector<PR>(SZ(A),{-1,-1}));
#define E(x,y) E[f.x][f.y]
  vector<F> FS;
  auto mf=[&](int i,int j,int k,int l){
    P3 q=(A[j]-A[i]).cross((A[k]-A[i]));
    if(q.dot(A[l])>q.dot(A[i]))q=q*-1;
    F f{q,i,j,k};
    E(a,b).ins(k);E(a,c).ins(j);E(b,c).ins(i);
    FS.push_back(f);
  };
  for(int i=0;i<4;i++)for(int j=i+1;j<4;j++)
    for(int k=j+1;k<4;k++)mf(i,j,k,6-i-j-k);
  for(int i=4;i<SZ(A);i++){
    for(int j=0;j<SZ(FS);j++){
      F f=FS[j];
      if(f.q.dot(A[i])>f.q.dot(A[f.a])){
        E(a,b).rem(f.c);E(a,c).rem(f.b);E(b,c).rem(f.a);
        swap(FS[j--],FS.back());FS.pop_back();
      }
    }
    int nw=SZ(FS);
    for(int j=0;j<nw;j++){
      F f=FS[j];
#define C(a,b,c) if(E(a,b).cnt()!=2)mf(f.a,f.b,i,f.c);
      C(a,b,c);C(a,c,b);C(b,c,a);
    }
  }
  for(F& it:FS)if((A[it.b]-A[it.a]).cross(
    A[it.c]-A[it.a]).dot(it.q)<=0)swap(it.c,it.b);
  return FS;
}
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

611f07, 8 lines

```
double sphericalDistance(double f1,double t1,
    double f2,double t2,double radius){
  double dx=sin(t2)*cos(f2)-sin(t1)*cos(f1);
  double dy=sin(t2)*sin(f2)-sin(t1)*sin(f1);
  double dz=cos(t2)-cos(t1);
  double d=sqrt(dx*dx+dy*dy+dz*dz);
  return radius*2*asin(d/2);
}
```

Miscellaneous (15)

integrate.hpp

Description: find definite integral with Simpson's method, error proportional to dx^4.

466ebb, 13 lines

```
template<class F>
db quad(db a,db b,const F &f,int n){
    db res=0;
    db dx=(b-a)/n;
    db fl=0,fr=f(a);
    for(int i=0;i<n;i++){
        db l=a+dx*i,r=l+dx;
        fl=fr,fr=f(r);
        db fm=f((l+r)/2);
        res+=fl+4*fm+fr;
    }
    return res*dx/6;
}
```

range-xor.hpp

Description: find all range of x such that l <= x xor p < r.

362b78, 8 lines

```
template<class F>
void range_xor(ll p,ll l,ll r,const F &query){
    for(ll b=1;l<r;b<<=1){
        if(l&b)query(l^p,(l^p)+b),l+=b;
        if(r&b)r-=b,query(r^p,(r^p)+b);
        if(p&b)p^=b;
    }
}
```

range-inversion.hpp

Description: Online range inversion count. Set n and a[1..n] (compressed to values in 1..n), call build(), then query(L,R) counts inversions in a[L..R] (1-indexed, inclusive). Needs n < N; K = block size ($\sqrt{N}$).

Time: build $\mathcal{O}\left(N\sqrt{N}\right)$, query $\mathcal{O}\left(\sqrt{N}\right)$

c46490, 103 lines

```
const int N=1e5+5;
const int K=320;

int n,k;
int a[N];
pair<int,int> b[N],c[N];
int block[N],l[K],r[K],pre[N],suf[N];
ll dp[K][K];
int cnt[K][N]; // prefix counts <= n*K, fits in int

struct Fenwick{
```

```
  int t[N];
  void update(int i,int v){
    while(i<N)t[i]+=v,i+=i&-i;
  }
  int query(int i){
    int res=0;
    while(i>0)res+=t[i],i-=i&-i;
    return res;
  }
}f;

ll calc(vector<int> &l,vector<int> &r){
  ll res=0;
  int p=0;
  for(auto x:l){
    while(p<r.size()&&x>r[p])p++;
    res+=p;
  }
  return res;
}

ll query(int L,int R){
  int bl=block[L],br=block[R];
  vector<int> x,y;
  if(bl==br){
    for(int i=l[bl];i<=r[bl];i++){
      auto [val,id]=c[i];
      if(id<L)x.emplace_back(val);
      if(L<=id&&id<=R)y.emplace_back(val);
    }
    return pre[R]-(L==l[bl]?0:pre[L-1])-calc(x,y);
  }
  ll ans=suf[L]+dp[bl+1][br-1]+pre[R];
  for(int i=bl+1;i<br;i++)ans+=cnt[i][r[bl]]-cnt[i][L-1]+cnt[i
      ][R]-cnt[i][l[br]-1];
  for(int i=l[bl];i<=r[bl];i++){
    auto [val,id]=c[i];
    if(L<=id)x.emplace_back(val);
  }
  for(int i=l[br];i<=r[br];i++){
    auto [val,id]=c[i];
    if(id<=R)y.emplace_back(val);
  }
  return ans+calc(x,y);
}

void build(){
    for(int i=1;i<=n;i++){
        b[i]={a[i],i};
    }
  k=(n-1)/K+1;
  for(int i=1;i<=k;i++){
        l[i]=r[i-1]+1,r[i]=i*K;
    }
  r[k]=n;
  for(int i=1;i<=k;i++){
    sort(b+l[i],b+r[i]+1);
    for(int j=l[i];j<=r[i];j++){
          block[j]=i,c[j]=b[j];
        }
    int res=0;
    for(int j=l[i];j<=r[i];j++){
      f.update(a[j],+1);
      res+=f.query(n)-f.query(a[j]);
      pre[j]=res;
    }
    dp[i][i]=res;
    for(int j=l[i];j<=r[i];j++){
      suf[j]=res;
      f.update(a[j],-1);
      res-=f.query(a[j]-1);
    }
  }
  sort(b+1,b+n+1);
  for(int i=1;i<=k;i++){
    for(int j=1,p=l[i];j<=n;j++){
      auto [val,id]=b[j];
      while(p<=r[i]&&val>c[p].first)p++;
      if(id<l[i])cnt[i][id]=p-l[i];
    }
    for(int j=1,p=l[i];j<=n;j++){
      auto [val,id]=b[j];
      while(p<=r[i]&&val>=c[p].first)p++;
      if(id>r[i])cnt[i][id]=r[i]-p+1;
    }
    for(int j=1;j<=n;j++)cnt[i][j]+=cnt[i][j-1];
  }
  for(int sz=2;sz<=k;sz++){
    for(int i=1,j=sz;j<=k;i++,j++){
      dp[i][j]=dp[i+1][j]+dp[i][j-1]-dp[i+1][j-1]+cnt[j][r[i]]-
          cnt[j][l[i]-1];
    }
  }
}
```

15.1 Game Theory

Sprague-Grundy: for impartial games (normal play: no move = lose), $g(x) = \text{mex}\{g(y) : x \to y\}$, x is losing iff $g(x) = 0$. Sum of independent games: $g = g_1 \oplus \cdots \oplus g_k$. A move that splits a game into several adds their g by xor.

- **Nim:** $g(n) = n$, losing iff $\bigoplus a_i = 0$.
- **Misère Nim** (last move loses): if all $a_i \le 1$, first player wins iff the number of non-empty piles is even; otherwise same as normal Nim.
- **Subtraction game** (take $1..k$): $g(n) = n \bmod (k+1)$. Any finite subtraction set: g is eventually periodic, find the period by brute force.
- **Staircase Nim** (move any coins from step i to $i-1$, step 0 is out): losing iff xor of piles on odd steps $= 0$.
- **Moore's Nim$_k$** (take from up to k piles at once): losing iff for every bit, the number of a_i with that bit set is $\equiv 0 \pmod{k+1}$.
- **Wythoff** (two piles; take any from one pile, or the same from both): losing positions are $(\lfloor k\varphi \rfloor, \lfloor k\varphi \rfloor + k)$, $\varphi = \frac{1+\sqrt{5}}{2}$.
- **Lasker's Nim** (take, or split a pile in two): $g(4k+1) = 4k+1$, $g(4k+2) = 4k+2$, $g(4k+3) = 4k+4$, $g(4k+4) = 4k+3$.
- **Green Hackenbush** (cut any edge, everything no longer connected to the root/ground is removed): for a tree rooted at the ground, $g(v) = \bigoplus_{c \in \text{children}(v)} (g(c)+1)$, $g(\text{leaf}) = 0$, answer $g(\text{root})$. A path of n edges is a Nim pile n. With cycles: first contract each cycle, an even cycle becomes one vertex, an odd cycle becomes one vertex with a loop edge (loop $= 1$).
- **Coin turning** (flip a set of coins whose rightmost goes heads to tails): g of a position = xor of g of each heads coin taken alone.